

Learning the RoPEs: Better 2D and 3D Position Encodings with STRING

Connor Schenck^{*†1}, Isaac Reid^{*^†23}, Mithun George Jacob^{*†1}, Alex Bewley^{*†1}, Joshua Ainslie^{*†1}, David Rendleman^{*†1}, Deepali Jain^{*1}, Mohit Sharma^{*1}, Avinava Dubey^{*3}, Ayzaan Wahid¹, Sumeet Singh¹, René Wagner¹, Tianli Ding¹, Chuyuan Fu¹, Arunkumar Byravan¹, Jake Varley¹, Alexey Gritsenko¹, Matthias Minderer¹, Dmitry Kalashnikov¹, Jonathan Tompson¹, Vikas Sindhwani¹, Krzysztof Choromanski^{*‡1}

¹Google DeepMind

²University of Cambridge

³Google Research

We introduce STRING: Separable Translationally Invariant Position Encodings. STRING extends Rotary Position Encodings [RoPE; 43], a recently proposed and widely used algorithm in large language models, via a unifying theoretical framework. Importantly, STRING still provides exact translation invariance, including token coordinates of arbitrary dimensionality, whilst maintaining a low computational footprint. These properties are especially important in robotics, where efficient 3D token representation is key. We integrate STRING into Vision Transformers with RGB(-D) inputs (color plus optional depth), showing substantial gains, e.g. in open-vocabulary object detection and for robotics controllers. We complement our experiments with a rigorous mathematical analysis, proving the universality of our methods. Videos of STRING-based robotics controllers can be found here: <https://sites.google.com/view/string-robotics>.

Keywords: Position Encoding, Transformers, Translation Invariance, Attention

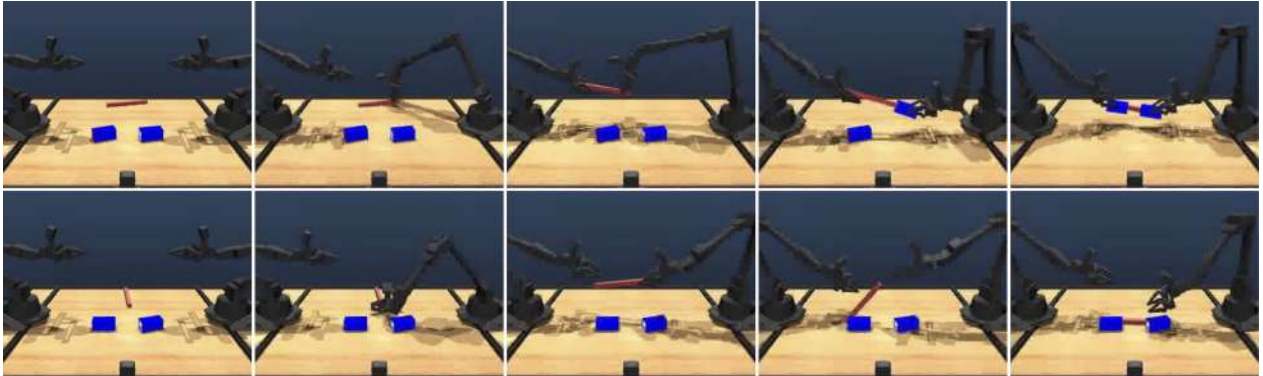


Figure 1 | **Top:** Successful diffusion policy conditioned on a STRING-enhanced Transformer vision encoder, attempting the double-insertion task on Aloha-sim. **Bottom:** Same experiment, but with a regular vision encoder for which the policy fails. STRING provides strong improvements for training dexterous robotics policies, outperforming previous position encoding algorithms such as RoPE.

1. Introduction and Related Work

Position encodings (PEs) [6, 25, 26, 53] inject information about the respective locations of tokens into transformers [48]. They are essential for good performance because vanilla attention is a set function, equivariant under permutation. In contrast, the meaning of a sequence of tokens in general depends on its ordering.

APEs and RPEs. Practitioners initially relied on *absolute* PEs [APEs; 26, 30, 48, 49] which add or concatenate fixed, precomputed position embeddings to tokens. These have since been replaced by *relative* PEs [RPEs; 8, 9, 28, 34, 36, 41], which add a learnable bias term that depends on the distance

between tokens to the pre-softmax attention logits. RPEs tend to generalise better than APEs over varying sequence lengths. However, they often require explicit computation for every query-key pair.

RoPE. To address the limitations of RPEs and APEs, researchers recently introduced *rotary* position encodings [RoPE; 22, 43]. These have been widely adopted in large language models [LLMs; 17, 18]. RoPE acts on queries and keys by partitioning them into 2-dimensional blocks, each of which is rotated by an angle proportional to the token’s position in the sequence. Whilst queries and keys are rotated separately, the angle of *relative* rotation is proportional to their separation, combining the best properties of APEs and RPEs. Mathematically, for query and key of dimensionality d , RoPE involves $\lfloor \frac{d}{2} \rfloor$ Givens rotations [3] acting on disjoint 2D subspaces.

Besides providing strong empirical gains, two attractive properties have driven the enthusiastic uptake of RoPE.

1. **Separability.** RoPE transforms each query and key independently, based on its position. This happens once per token; the PE’d tokens are not recalculated during subsequent processing like autoregressive generation. This makes KV-caching convenient. Separability also makes RoPE compatible with linear attention, e.g. Performers [10, 24]. Here, the attention matrix is not instantiated in memory so explicit RPE mechanisms are not possible.²
2. **Translational invariance.** For a query-key pair at positions $(i, j) \in \mathbb{N}^2$, the relative rotation angle depends only on $i - j$. This improves sequence-length generalization.

However, RoPE is not the only position encoding algorithm with these desirable traits. In this paper, we propose a more general algorithm called **STRING: Separable Translationally Invariant Position Encodings**. STRING is based on Lie groups. It generalises RoPE via a unifying theoretical framework, incorporating the latter as a special case. In fact, we later prove that STRING is the *most general* PE algorithm with the properties above, amongst a broad class.

STRING for robotics. The above features are especially important in robotics, where efficient 2D/3D token representation and sensible physical priors are key. To demonstrate it, we integrate STRING into Vision Transformers (ViTs), showing strong improvements for open-vocabulary object detection models and various robotics controllers. This showcases the real-world impact of our STRING.

Videos of STRING-based robotics controllers can be found here: <https://sites.google.com/view/string-robotics>.

Key contributions.

1. We introduce STRING, a new family of position encodings for multidimensional token coordinates that respect both separability and translational invariance.
2. We rigorously analyse STRING’s theoretical properties (**Sec. 3**), proving that it is more general than RoPE. We provide computationally efficient implementations.
3. We show strong accuracy gains across varied models using Transformers with STRING, on a range of robotics and general vision tasks (see Fig. 1 and **Sec. 4**).

2. Preliminaries

Let $\{\mathbf{x}_i\}_{i=1}^N \in \mathbb{R}^d$ denote a set of N d -dimensional tokens. Assume that d is even. The i th query, key and value vectors are given by $\mathbf{q}_i = \mathbf{W}_q \mathbf{x}_i$, $\mathbf{k}_i = \mathbf{W}_k \mathbf{x}_i$ and $\mathbf{v}_i = \mathbf{W}_v \mathbf{x}_i$ respectively, where $\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v \in \mathbb{R}^{d \times d}$ are learned projection matrices (to keep the notation simple, we assume here the one-head setting). The *attention mechanism*, basic computational unit of the Transformer, can be written as:

²*Implicit* relative position encoding schemes, which avoid instantiating the attention matrix in memory, have also been proposed [11, 31, 37].

$$\mathbf{x}_i \rightarrow \frac{\sum_j \exp(\mathbf{q}_i^\top \mathbf{k}_j) \mathbf{v}_j}{\sum_l \exp(\mathbf{q}_i^\top \mathbf{k}_l)}. \quad (1)$$

This updates the set of tokens, mixing them dynamically depending on the query-key softmax similarity scores.

Rotary position encodings. As described in Section 1, RoPE rotates tokens depending on their locations. In the 1D data setting (e.g. text), for a token $\mathbf{z}_i \in \{\mathbf{q}_i, \mathbf{k}_i\}$ at position $i \in \mathbb{N}$, we take $\mathbf{z}_i \rightarrow \text{RoPE}(i)\mathbf{z}_i$ with

$$\text{RoPE}(i)\mathbf{z}_i := \bigoplus_{n=1}^{d/2} \boldsymbol{\rho}(i\theta_n)[\mathbf{z}_i]_{2n-1:2n}, \quad (2)$$

$$\boldsymbol{\rho}(\theta) := \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}. \quad (3)$$

Here, $\bigoplus$ denotes the direct product, so that each 2×2 matrix $\{\boldsymbol{\rho}(i\theta_n)\}_{i=1}^{d/2}$ independently rotates a 2-element section of the query/key, and $[\mathbf{z}_i]_{2n-1:2n}$ denotes the $2n-1$ and $2n$ elements of $\mathbf{z}_i$. Note that the matrix $\text{RoPE}(i)$ is $d \times d$, but it is only nonzero on the 2×2 blocks on the diagonal. Since $\boldsymbol{\rho}(\theta)^\top = \boldsymbol{\rho}(-\theta)$ and 2D rotations commute, we have that

$$\text{RoPE}(i)^\top \text{RoPE}(j) = \text{RoPE}(j-i), \quad (4)$$

whereupon we are transforming $\mathbf{q}_i^\top \mathbf{k}_j \rightarrow \mathbf{q}_i^\top \text{RoPE}(j-i)\mathbf{k}_j$. The dependence on $j-i$ makes this translationally invariant. RoPE takes the set of angles $\{\theta_n\}_{n=1}^{d/2}$, determining the rotation frequency of each 2×2 block, as hyperparameters. We suppress this dependence for notational compactness. The authors originally proposed the decaying sequence $\theta_n = \lambda^{-2(n-1)/d}$ with base wavelength $\lambda = 10,000$, though variants have since been explored (see below).

RoPE in higher dimensions. Whilst RoPE was originally proposed for sequence data, recent work has extended it to encode higher-dimensional position information [22]. Now each token is equipped with a vector $\mathbf{r}_i \in \mathbb{R}^{d_c}$, and we require: $\text{RoPE}(\mathbf{r}_i)^\top \text{RoPE}(\mathbf{r}_j) = \text{RoPE}(\mathbf{r}_j - \mathbf{r}_i)$. Since 2D rotations commute, one approach is to define

$$\text{RoPE}(\mathbf{r}_i) := \prod_{k=1}^{d_c} \text{RoPE}([\mathbf{r}_i]_k), \quad (5)$$

where $[\mathbf{r}_i]_k$ is the k th coordinate of $\mathbf{r}_i$ (with $k \in \{1, \dots, d_c\}$). This independently applies regular 1-dimensional RoPE (Eq. 2) for each dimension of the position vector. The rotation frequencies $\{\theta_n\}_{n=1}^{d/2}$ can optionally differ between each coordinate axis.

Generalisations of RoPE. Prompted by its success, a number of papers have since sought to understand the effectiveness of RoPE and propose better-performing alternatives. One well-known method argues to increase the base wavelength λ to 500,000, slowing the rate of token rotation and improving learning with longer contexts [38, 50]. Another suggests to completely truncate the lowest frequencies, setting them to zero, which helps preserve long-range ‘semantic channels’ [1]. Practitioners can also make the parameters $\{\theta_n\}_{n=1}^{d/2}$ fully learnable, improving flexibility. Lastly, recent work has proposed to replace the block-diagonal RoPE matrices $\text{RoPE}(i)$ by more general dense matrices in $\text{SO}(d)$, parameterized by learned antisymmetric generators [33]. Whilst more expressive, this algorithm breaks translational invariance for position vectors with $d_c > 1$, and has a large memory footprint. This makes it unsuitable for robotics applications. In Section 3, we will propose a better alternative, STRING.

3. STRING: Separable Translationally Invariant Position Encodings

Recall that our goal is modify queries/keys depending on their respective positions, so that changes to dot products $\mathbf{q}_i^\top \mathbf{k}_j$ depend on $\mathbf{r}_i - \mathbf{r}_j$. RoPE achieves this using matrix multiplication [43]. Here, we present STRING: a more general, better-performing algorithm.

3.1. Introducing STRING

STRING is defined as follows.

Definition 3.1. STRING is the mapping $\mathbf{R}(\cdot) : \mathbb{R}^{d_c} \rightarrow \mathbb{R}^{d \times d}$, from d_c -dimensional position vectors to $d \times d$ matrices, given by

$$\mathbf{R}(\mathbf{r}_i) = \exp \left(\sum_{k=1}^{d_c} \mathbf{L}_k [\mathbf{r}_i]_k \right), \quad (6)$$

where $\{\mathbf{L}_k\}_{k=1}^{d_c} \subset \mathbb{R}^{d \times d}$ is a set of learnable and commuting skew-symmetric generators. Given a set of queries or keys $\{\mathbf{z}_i\}_{i=1}^N \subset \mathbb{R}^d$ at positions $\{\mathbf{r}_i\}_{i=1}^N \subset \mathbb{R}^{d_c}$, their positions are encoded as:

$$\mathbf{z}_i \rightarrow \mathbf{R}(\mathbf{r}_i) \mathbf{z}_i \quad \forall i \in \{1, \dots, N\}. \quad (7)$$

Here, $\exp(\cdot)$ refers to the *matrix* exponential, defined by its series expansion $\exp(\mathbf{A}) := \sum_{i=0}^{\infty} \mathbf{A}^i / i!$ and $[\mathbf{r}_i]_k$ is the k th coordinate of vector $\mathbf{r}_i$. By ‘commuting skew-symmetric generators’, we mean that $\{\mathbf{L}_k\}_{k=1}^{d_c}$ satisfy

$$[\mathbf{L}_i, \mathbf{L}_j] = 0 \quad \text{and} \quad \mathbf{L}_i^\top = -\mathbf{L}_i \quad \forall i, j. \quad (8)$$

There are many ways to parameterize such a set; we give examples in Section 3.2. Remarkably, the following is true.

Theorem 3.2 (STRING is general). *Consider the set of mappings $\mathbf{R}(\cdot) : \mathbb{R}^{d_c} \rightarrow \mathbb{R}^{d \times d}$ that satisfy the group-like translational invariance property $\mathbf{R}(\mathbf{r}_i)^\top \mathbf{R}(\mathbf{r}_j) = \mathbf{R}(\mathbf{r}_j - \mathbf{r}_i) \forall \mathbf{r}_i, \mathbf{r}_j \in \mathbb{R}^{d_c}$, are continuously differentiable with respect to $\mathbf{r}_i$, and satisfy $\mathbf{R}(\mathbf{0}) = \mathbf{I}_d$ (with $\mathbf{I}_d$ the d -dimensional identity). All such mappings can be expressed as STRING with some set of generators $\{\mathbf{L}_k\}_{k=1}^{d_c} \subset \mathbb{R}^{d \times d}$.*

In this sense, STRING is the *most general* of all translationally invariant position encoding mechanisms using matrix multiplication. Meanwhile, RoPE is a simple special case of STRING, taking a particular choice of generators. This can be seen as follows.

Theorem 3.3 (RoPE is a type of STRING #1). *Consider the generators $\mathbf{L}_k = \sum_{p=1}^{d/2} (\delta_{2p, 2p-1} - \delta_{2p-1, 2p}) \theta_p$, where $\{\theta_p\}_{p=1}^{d/2} \subset \mathbb{R}$ and $\delta_{i,j}$ is the delta function. This corresponds to RoPE with rotational frequencies $\{\theta_p\}_{p=1}^{d/2}$.*

Proofs of Theorem 3.2 and Theorem 3.3 are in Appendix A.

3.2. Computationally efficient STRING

Despite being general and notationally compact, the parameterization of the STRING matrices $\mathbf{R}(\mathbf{r}_i)$ shown in Eq. 6 may not be convenient for practical applications. Given N tokens at positions $\{\mathbf{r}_i\}_{i=1}^N$, one must in general exponentiate and store N dense $d \times d$ matrices. This incurs $O(Nd^3)$ time complexity and $O(Nd^2)$ space complexity cost. The problem is exacerbated if the $\{\mathbf{r}_i\}_{i=1}^N$ differ across training examples and batches, which occurs e.g for point cloud data or color plus depth channel (RGB-D) images. In this case, $\{\mathbf{R}(\mathbf{r}_i)\}_{i=1}^N$ cannot be cached and reused. This motivates the goal of this section:

to find *efficient* STRING instantiations, nonetheless more general and expressive than RoPE. We begin with the following (proof in Appendix A):

Theorem 3.4 (RoPE is a type of STRING #2). *For any STRING position encoding with generators $\{\mathbf{L}_k\}_{k=1}^{d_c}$, there exists an orthogonal matrix $\mathbf{P}$ such that*

$$\mathbf{R}(\mathbf{r}_i) = \mathbf{P}\text{RoPE}(\mathbf{r}_i)\mathbf{P}^\top. \quad (9)$$

Note that the orthogonal matrix $\mathbf{P}$ is independent of the coordinates $\mathbf{r}_i$, so it can be learned and stored once per attention head and shared across all training examples. Meanwhile, $\text{RoPE}(\mathbf{r}_i)$ is sparse – it is only nonzero on the super- and subdiagonals – so multiplying tokens only requires $O(Nd)$ memory and $O(Nd^2)$ time, saving a factor of d . This is crucial in the contrastive learning setting where batch sizes can become large. Once again, one can see that RoPE is a special case of STRING, this time taking $\mathbf{P} = \mathbf{I}_d$. We emphasize that the parameterization of STRING in Eq. 9 remains just as general as in Eq. 6. We also note that, since in regular attention one takes dot products between position-encoded queries and keys, the first orthogonal matrix $\mathbf{P}$ will always cancel with its counterpart. Therefore, in Transformers it is sufficient to take $\mathbf{R}(\mathbf{r}_i) = \text{RoPE}(\mathbf{r}_i)\mathbf{P}$, with $\mathbf{P} \in O(d)$ learnable, without loss of generality.³

Example 1: Cayley-STRING. Equipped with Theorem 3.4, our goal becomes to choose a suitable parameterization for the orthogonal matrix $\mathbf{P}$. One option is to take the *Cayley Transform*,

$$\mathbf{P}_{\text{Cayley}} := (\mathbf{I}_d - \mathbf{S})(\mathbf{I}_d + \mathbf{S})^{-1}, \quad (10)$$

where $\mathbf{S}$ is a learnable (potentially sparse) antisymmetric matrix [14]. $\mathbf{P}_{\text{Cayley}}$ is convenient since, for a token $\mathbf{z}_i$, we can compute $(\mathbf{I}_d + \mathbf{S})^{-1}\mathbf{z}_i$ efficiently using a linear solver, avoiding matrix inverse computation. Where we use $\mathbf{P}_{\text{Cayley}}$, we refer to our algorithm as *Cayley-STRING*.

The unreasonable effectiveness of STRING. In some sense, Theorem 3.4 makes it surprising that STRING outperforms RoPE so comprehensively in all our experiments (see Section 4), given that they are related by a change of basis. It appears that the ability to *explicitly* learn this basis change via $\mathbf{P}$ (shared between queries and keys), rather than implicitly via existing network weights, substantially boosts performance. Conversely, when using linear attention variants, the projected tokens $\mathbf{W}_q\mathbf{q}_i$ and $\mathbf{W}_k\mathbf{k}_i$ are pushed through nonlinearities such as $\text{ReLU}(\cdot)$ before taking the dot product. Hence, in this case, including learnable $\mathbf{P}$ does increase the capacity of the network, rather than simply learning a basis change.

Example 2: Circulant-STRING. We now present a second efficient STRING algorithm within our framework. A square matrix is referred to as *circulant* if it takes the form

$$\mathbf{C} = \begin{bmatrix} c_0 & c_{d-1} & \cdots & c_2 & c_1 \\ c_1 & c_0 & c_{d-1} & \cdots & c_2 \\ \vdots & c_1 & c_0 & \ddots & \vdots \\ c_{d-2} & \vdots & \ddots & \ddots & c_{n-1} \\ c_{d-1} & c_{d-2} & \cdots & c_1 & c_0 \end{bmatrix}. \quad (11)$$

All rows are composed of the same elements, and each row is rotated one element relative to the preceding row. The transpose of a circulant matrix $\mathbf{C}^\top$ is also circulant, and the sum of two circulant matrices is also circulant. It follows that $\mathbf{C} - \mathbf{C}^\top$ is circulant and antisymmetric. Lastly, circulant matrices commute. With these properties in mind, we can simply define $\mathbf{L}_k = \mathbf{C}_k - \mathbf{C}_k^\top$ for $k \in \{1, \dots, d_c\}$, with $\mathbf{C}_k$ a learnable circulant matrix parameterized by d scalars $\{c_0, \dots, c_{d-1}\}$. We call this *Circulant-STRING*. This special parameterization is convenient for the following reason.

³We dropped the transpose sign on the second $\mathbf{P}$, redefining $\mathbf{P}^\top$ as our learnable orthogonal matrix.

Theorem 3.5 (Circulant-STRING is fast). *Given generators $\mathbf{L}_k = \mathbf{C}_k - \mathbf{C}_k^\top$ with $\mathbf{C}_k$ circulant, the position encoding $\exp(\sum \mathbf{L}_k[\mathbf{r}_i]_k)\mathbf{z}_i$ for token $\mathbf{z}_i$ at position $\mathbf{r}_i$ can be computed in $O(d \log d)$ time and $O(d)$ memory using the fast Fourier Transform (FFT).*

We provide a proof in Appendix A. Circulant-STRING provides another efficient position encoding algorithm that scales gracefully to large, high-dimensional datasets and performs well in spatial applications (see Section 4).

Learnable frequencies with STRING. Note that the STRING generators from Definition 3.1 are (in general) learnable. For Cayley-STRING, the angle-defining frequencies for RoPE and $\mathbf{S}$, the antisymmetric matrix from Equation (10) are learned whereas in Circulant-STRING, the scalars $\{c_0, \dots, c_{d-1}\}$ in Equation (11) are learned.

3.3. Loose ends

Here, we discuss further generalisations of STRING.

Extension 1: $\otimes$ -STRING. So far, we have followed RoPE in assuming that our position encodings are applied via matrix multiplication. However, this can be relaxed whilst preserving separability and translational invariance. For example, one can transform tokens $\mathbf{z}_i$ via the *outer* product with position feature vectors $\mathbf{f}(\mathbf{r}_i) \in \mathbb{R}^{2m}$,

$$\mathbf{z}_i \rightarrow \text{vec}(\mathbf{f}(\mathbf{r}_i) \otimes \mathbf{z}_i). \quad (12)$$

Here, $\otimes$ denotes the outer product and vec denotes the ‘vectorizing’ operation that flattens a matrix to a vector, so that $\text{vec}(\mathbf{f}(\mathbf{r}_i) \otimes \mathbf{q}_i)_{da+b} = \mathbf{f}(\mathbf{r}_i)_a \mathbf{q}_i_b$ where $a \in \{1, \dots, 2m\}$ and $b \in \{1, \dots, d\}$. Since the dot product of (flattened) outer products gives the product of dot products, we have

$$\text{vec}(\mathbf{f}(\mathbf{r}_i) \otimes \mathbf{q}_i)^\top \text{vec}(\mathbf{f}(\mathbf{r}_j) \otimes \mathbf{k}_j) = \mathbf{q}_i^\top \mathbf{k}_j \cdot \mathbf{f}(\mathbf{r}_i)^\top \mathbf{f}(\mathbf{r}_j). \quad (13)$$

Now suppose that we take the Fourier features

$$\mathbf{f}(\mathbf{r}_i) = \frac{1}{\sqrt{m}} \left[\cos(\boldsymbol{\omega}_k^\top \mathbf{r}_i), \sin(\boldsymbol{\omega}_k^\top \mathbf{r}_i) \right]_{k=1}^m, \quad (14)$$

where $\{\boldsymbol{\omega}_k\}_{k=1}^m \subset \mathbb{R}^d$ are learnable d -dimensional frequency vectors. Then we have that $\mathbf{f}(\mathbf{r}_i)^\top \mathbf{f}(\mathbf{r}_j) = \frac{1}{m} \sum_{k=1}^m \cos(\boldsymbol{\omega}_k(\mathbf{r}_i - \mathbf{r}_j))$ which is clearly a function of $\mathbf{r}_i - \mathbf{r}_j$. We refer to this novel position encoding variant, orthogonal to previous RoPE-like approaches, as $\otimes$ -STRING.

Extension 2: General transformation groups. Having focused on *translational* invariance, another natural question is whether STRING could be repurposed for other continuous transformation groups. These may be more suitable for data with different properties; for example, one might sometimes prefer a *rotationally* invariant position encoding.

More formally, recall that a Lie group with parameters $\psi \in \mathbb{R}^k$ is a group of transformations of the form $T_\psi : \mathbb{R}^d \rightarrow \mathbb{R}^d$ that are differentiable with respect to ψ . Let the parameter $\psi = 0$ correspond to the identity element, so that $T_0 \mathbf{x} = \mathbf{x}$. A *canonical coordinate system* for G is an injective map ρ from Cartesian coordinates to a new coordinate system, satisfying $\rho(T_\psi \mathbf{x}) = \rho(\mathbf{x}) + \sum_{i=1}^k \psi_i \mathbf{e}_i \forall T_\psi \in G$, where $\mathbf{e}_i$ is the i th standard basis vector. Observe that the right hand side of this equation represents a translation in the new basis. Canonical coordinate systems exist for all one-parameter Lie groups ($k = 1$), and more generally for Abelian groups of dimension $k \leq d$ [39, 40, 44]. They can be derived analytically by solving a set of first-order PDEs, though for many common transformation groups the canonical coordinate system is obvious. For instance, for azimuthal and polar rotations of points $(\mathbf{r}_x, \mathbf{r}_y, \mathbf{r}_z)$ in 3D space ($k = 2$), a canonical coordinate system is (θ, ϕ) , where $\sin \theta := \sqrt{\mathbf{r}_x^2 + \mathbf{r}_y^2} / \|\mathbf{r}\|_2$

and $\tan \phi := r_y/r_x$. Rotating⁴ simply ‘translates’ the canonical coordinates $(\theta, \phi) \rightarrow (\theta + \Delta\theta, \phi + \Delta\phi)$ – a transformation looking much more complicated in the Cartesian basis.

STRING for Abelian Lie groups. It follows that, simply by replacing Cartesian coordinates $\{r_i\}_{i=1}^N$ with their canonical counterparts, we can repurpose STRING to construct position encodings that respect more general invariances.

4. Experiments

In this section, we provide an exhaustive empirical comparison of STRING with RoPE and vision encoders leveraging regular APEs. To set up the ground, we start with general non-robotics experiments in Sec. 4.1. On our way to robotics applications, we then test STRING for 2D and 3D object detection in Sec. 4.2. Finally, we present robotics manipulation experiments in Sec. 4.3 and Sec. 4.4.

4.1. General Experiments: Classification and Retrieval

We tested STRING for image classification tasks on the ImageNet2012 [13] and Places365 datasets, with Vision Transformer (ViT) [15] as our base model. We compare against RoPE and RoPE-Mixed [22], abbreviated to RoPE-M to Circulant-STRING and Cayley-STRING (respectively abbreviated to Circulant-S and Cayley-S). The results are shown in Table 1. For both datasets, STRING offers best models. For ImageNet2012, top two models are STRINGs. Furthermore, for ImageNet2012 STRINGs provide the first absolute gains larger than 1%, as compared to regular ViTs, with only a negligible set of extra trainable parameters.

	ViT	RoPE	RoPE-M	Circulant-S	Cayley-S
ImageNet	80.04	80.18	80.86	81.22	<u>81.09</u>
Places365	56.79	<u>56.97</u>	56.69	56.77	57.16
Mean	68.42	68.58	68.78	<u>69.00</u>	69.12

Table 1 | Image classification % test accuracy. Best numbers are highlighted in **bold** and the second-best numbers are underlined.

Next, we lift WebLI [7], a dataset of 10 billion image-text pairs across a variety of languages, into 3D by pre-processing a 60-million image subset with Depth-Anything-V2 [51] for metric mono-depth estimation. The dataset is filtered using methods from [5] for images that the indoor-finetuned model performs poorly on (pictures with overlays, no visible groundplane, large outdoor scenes, optical illusions are removed).

We perform contrastive learning on the text to visual representation pairs in the WebLI-3D lifted dataset, where the visual representation may be in the form of an RGB image or an RGB-D depth image. Similar to CLIP [35], this is done by maximizing the similarity between the embeddings of matching visual-text pairs, while minimizing the similarity between embeddings of the non-matching pairs. This enables open-vocabulary detection and classification by comparing the text embeddings of all possible classes against those of the visual representation, and selecting the minimum distance pair. We compare against baseline in Table 2. **For all six evaluations**, Cayley-STRING is the best and Circulant-STRING is second best.

⁴Note that this differs from full 3D object pose invariance, for which the transformations do *not* form an Abelian group.

	i2t@1	i2t@5	i2t@10	t2i@1	t2i@5	t2i@10	Mean
ViT	53.88	73.17	78.49	53.98	73.83	79.29	68.77
RoPE	55.27	74.27	79.52	55.22	74.61	80.25	69.86
RoPE-M	55.30	74.08	79.47	55.36	74.73	80.18	69.85
Circulant-S	<u>55.52</u>	<u>74.69</u>	<u>79.91</u>	<u>55.68</u>	<u>75.03</u>	<u>80.45</u>	<u>70.21</u>
Cayley-S	55.70	75.08	80.24	55.82	75.40	80.65	70.48

Table 2 | Image-to-text (i2t) and text-to-image (t2i) WebLI recall @ rank (best numbers: in **bold**, second-best: underlined.)

4.2. Improving Open-Vocabulary Object Detection

4.2.1. Open-Vocabulary Object Detection in 2D

We demonstrate the efficacy of STRING on open-vocabulary object detection for localizing a 2D bounding box on standard RGB image benchmarks. For a baseline, we use the official implementation of OWL-ViT⁵ [32] which applies a standard Vision Transformer [15] with light-weight classification and localization heads for detection. Table 3 compares the baseline OWL-ViT model with RoPE and STRING variants. For all experiments, we followed the standard training pipeline for the B/32 backbone with the CLIP loss [35]. Even in this axis-aligned 2D detection setting – which is favourable for the standard RoPE variant – Cayley-STRING provides the best overall performance.

	Baseline	RoPE	RoPE-M	Circulant-S	Cayley-S
AP ^{COCO}	32.44	33.66	32.74	33.24	<u>33.47</u>
AP ^{LVIS}	21.98	<u>22.71</u>	22.43	22.60	23.01
Mean	27.21	<u>28.18</u>	27.59	27.92	28.24

Table 3 | Average Precision (AP) % of the OWL-ViT model on COCO [29] and LVIS [19]. Best in **bold**, second-best underlined.

4.2.2. Open-Vocabulary Object Detection in 3D

We tested STRING on the open-vocabulary 3D object bounding box prediction task, similar to those from Section 4.2.1. Here we modify the output to be the full SE(3) pose and 3D size of the bounding box. We initialize the weights of the vision and text towers of our model with the weights from the models trained on the WebLI-3D dataset from Section 4.1. We replace the IOU loss from OWL-ViT with an 8-corner vertex loss, but otherwise keep the same matching and loss algorithm. We train on a simulated dataset of 4 million images of indoor and tabletop scenes with groundtruth 3D bounding box labels (see Appendix F.1). From this dataset, we hold out 80 images for evaluation. We evaluate both ViT and ViTD variants. The 3D intersection-over-union (IOU) values for various RoPE and STRING variants on the evaluation data are shown in Table 4. For each configuration, we train 3 models with different random seeds and take the best performing model (see Appendix F.2.4 for details). Fig. 2 shows example 3D detections for 6 different variants (see Appendix F for details). Note that STRINGs provide much more accurate prediction of the 3D bounding boxes for more challenging to localize smaller objects than baseline and RoPE. For ViT, Circulant-STRING is the best, providing 1.5% relative improvement as compared to the best RoPE variant. For ViTD, Cayley-STRING is the

⁵https://github.com/google-research/scenic/tree/main/scenic/projects/owl_vit

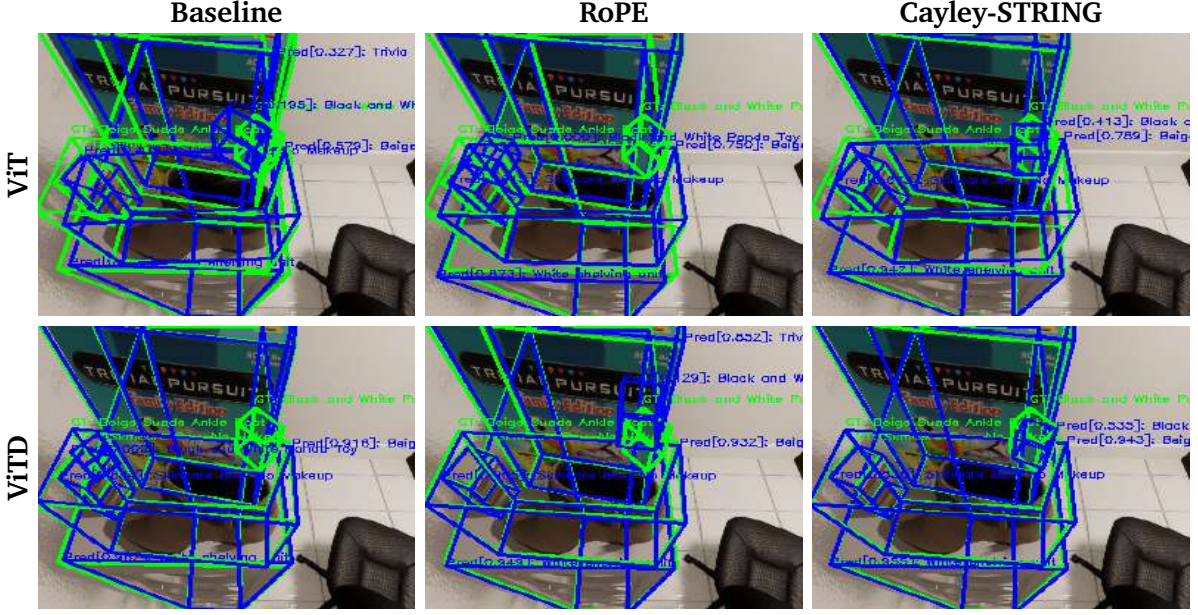


Figure 2 | Example outputs for the 3D detection task for baseline, RoPE, and Cayley-S. Green boxes: groundtruth. Blue boxes: predictions.

best, providing 2% relative improvement as compared to the best RoPE variant. For both ViT and ViTD, two best models are STRINGs.

	Baseline	RoPE	RoPE-M	Circulant-S	Cayley-S
ViT	49.77	58.09	57.17	58.95	58.85
ViTD	67.60	71.21	70.90	<u>72.36</u>	72.67

Table 4 | Average 3D IOU % over all objects for the 3D bounding box prediction task. For each setting, 3 models were trained with different random seeds and the max is reported. Baseline indicates no RoPE or STRING. Best in **bold**, second-best underlined.

4.3. Simulation-Based Robot Manipulation: ALOHA

We evaluate the performance of STRING on dexterous robotics tasks using ALOHA 2, a bimanual parallel-jaw gripper workcell with two 6-DoF arms, within the ALOHA Unleashed [54] simulation environment (see: Fig. 1). ALOHA Unleashed utilizes a scalable teleoperation framework used to collect human demonstration data.

We trained ALOHA Unleashed’s Transformer-based neural network with diffusion policies (conditioned on vision encoders) on human demonstrations of 12 dexterous tasks (see Appendix B for descriptions and renders). The vision system utilized images from RGB cameras positioned overhead, on each wrist, and at the level of the table. We also deployed our policies on real ALOHA 2 robots (see Fig. 3 and Fig. 11). Due to the large observed variance of the on-robot evaluation for ALOHA 2, we focused

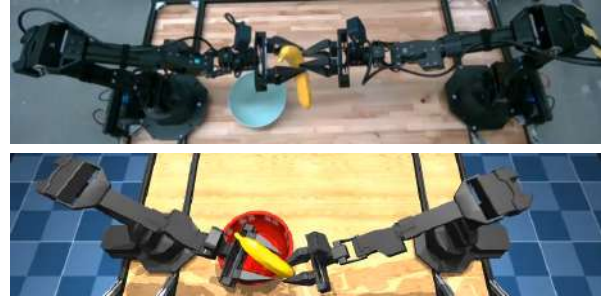


Figure 3 | **HandOverBanana** task for the ALOHA 2 robot: real (top) and the corresponding simulated (bottom) evaluation.

on the evaluation in simulation to accurately rank different methods.

Table 5 reports mean success rate (best in **bold**, second-best underlined) over 10 evaluations of each ALOHA simulation task. ALOHA leader robots are teleoperated in ALOHA simulation for data collection using an ALOHA station, an Oculus VR headset and controllers [54]. The teleoperators are instructed to perform the following tasks using this setup. RoPE [22] and Cayley-STRING are added to a baseline SigLIP B/16 256 ViT [52]. See Appendix B for details. We find that STRING has the best success rate across 11 of 13 tasks, thereby showcasing the effectiveness of our model

	ViT	RoPE	STRING
BowlOnRack	<u>0.90</u>	0.80	1.00
DoubleInsertion	<u>0.20</u>	<u>0.50</u>	0.60
FMB-1	0.20	0.20	0.20
FMB-2	0.10	0.10	0.10
FruitBowl	0.30	0.30	0.30
GlassOnRack	0.60	0.60	0.60
HandOverBanana	1.00	1.00	1.00
HandOverPen	1.00	1.00	1.00
MugOnPlate	0.70	0.90	<u>0.80</u>
PlateOnRack	<u>0.60</u>	0.70	0.50
SingleInsertion	<u>0.40</u>	0.60	0.60
StorageBin	0.00	0.00	0.00
MultiTask	0.37	<u>0.42</u>	0.46

Table 5 | Mean success rate on ALOHA simulation task.

The success rate is averaged over 10 trials of each checkpoint, taken every 10K train steps and over 1M train steps. Corresponding curves are given in Fig. 4. Cayley-STRING achieves superior results across all tasks on average (i.e. MultiTask). Additionally, it achieves equivalent or superior results, as compared to RoPE (e.g. for the DoubleInsertion task from Fig. 1) and ViT for all 12 tasks except for MugOnPlate and PlateOnRack (second-best). Finally, STRING converges much faster than other methods (see: Fig. 4). Note that we applied the strategy of learning all angle-defining frequencies for both RoPE and Cayley-STRING.

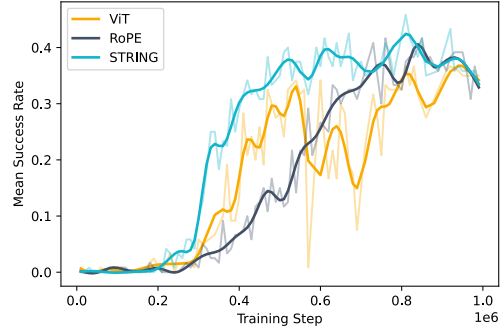


Figure 4 | Mean success rate across all tasks (i.e. MultiTask) evaluated 10 times every 10K train steps over 1M train steps.

4.4. Real-World 3D Robot Manipulation: KUKA

Establishing STRING as superior to other methods on previous tasks, we let it operate on 3D data to obtain new SOTA robotic manipulation policies. This resulted in policies directly using depth and deployed on real hardware. Note that STRING can be naturally applied in that context since it can be used for data equipped with general coordinate vectors $\mathbf{r}_i$ associated with tokens (e.g. 3D).

4.4.1. Setting

We evaluated STRING in the vision encoder of a generative policy applying energy-based models [42] and deployed on a real industrial KUKA robot arm [46]. The closed-loop feedback policy operates on the RGB-D images, and is learned as a generative model with imitation learning. Its architecture is shown in Figure 14 (Appendix G), and consists of the diffusion Transformer and the 3D encoder. The policy was trained on a mixture of scripted and teleoperated data collected for 3 different skills (pick, place and handover) on various objects. It is evaluated exclusively on the pick skill with a diverse set of objects. Each evaluation was run as an A/B test for a total of 50 trials.

4.4.2. Regular evaluations

We experimented with two ways of using depth in the policy.

Implicit depth via normal maps: In the first approach, following [45], depth input is used to construct a surface normal map with unit $\mathbb{R}^3$ values per pixel. Both RGB and depth inputs are then processed via identical (shared weights) embedding layers. The embeddings are concatenated and processed through Transformer layers. Finally, the embeddings are split and fused to yield the final vision embedding. Our results in Figure 5 show that this approach of incorporating depth has a detrimental effect on the on-robot deployed policy. We hypothesize that this is caused by the significant amount of noise coming from the depth sensors, leading to imperfect surface normal maps.

Lifting patches to 3D for STRING: In the

second approach, we compute the height for each patch via mean-pooling across depth values for all the pixels in the patch, followed by the learnable linear layer. The resulting 3D patches are then fed to the Transformer, with positional encoding given by STRING to incorporate depth into the vision encoder. Our results Figure 5 show that STRING improves the success rate over the 2D base policy. Also, when STRING is combined with the first method, it **drastically** reduces the negative impact of noisy normal maps.

We used Circulant-STRING to obtain a particularly compact computational footprint. Note that in this setting, more computationally intense mechanisms, such as [33], run out of memory and could not be trained.

4.4.3. Out-of-distribution evaluation: STRING vs baseline

To further compare STRING with the baseline and show the advantages of using 3D over 2D policies, we also perform *out-of-distribution* (OOD) evaluations on the real robot.

We vary three different environment settings. These include: (1) lighting changes, (2) adding large distractor objects and (3) changing the height of the table from which the robot has to grasp the block. For each setting, we test multiple different variations, e.g., three different light settings.

Figure 6 compares STRING with the 2D baseline for each OOD setting. For these evaluations, we choose the best policies from Section 4.4.2. As seen in Figure 6, 3D STRING policies outperform 2D policies across all OOD settings. For instance, with large distractors (middle), the 2D model’s performance decreases from 65% to 57%, while 3D STRING *maintains* performance similar to non-OOD settings ($\approx 74\%$). In some OOD cases, such as lighting changes, both 2D ($\approx 10\%$) and 3D ($\approx 25\%$) policies experience a performance decrease vs. the non-OOD setup. This drop in performance during lighting changes is likely due to the significant alteration in image observations, thus affecting both 2D and 3D policies. Finally, the largest performance difference is observed in the table height variation scenario. Here, the 3D policies exhibit significantly higher robustness ($\approx 50\%$) compared to the 2D policies ($\approx 10\%$). This suggests that the 3D STRING policy leverages the raw depth signal to better generalize to table height variations, a change imperceptible to fixed monocular cameras.

Overall, our results show that 3D STRING policies are highly robust to many variations and significantly improve over 2D policies. Fig. 7 shows a sample episode from the on-robot evaluation of the STRING generative policy.

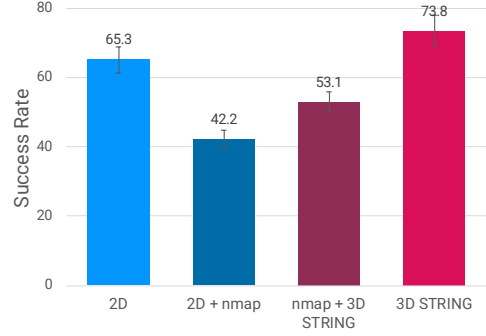


Figure 5 | Performance of STRING with 3D input vs. baselines on real-robot tasks (with 2 seeds). 2D baseline performance without depth input is $\approx 65\%$. Incorporating depth through surface normal maps (nmap) reduces performance to 42%. Using 3D STRING for incorporating depth improves the performance in both scenarios - with and without normal maps to 53% and 74% respectively. Mean/stdev shown above were calculated from 35 evaluation runs.

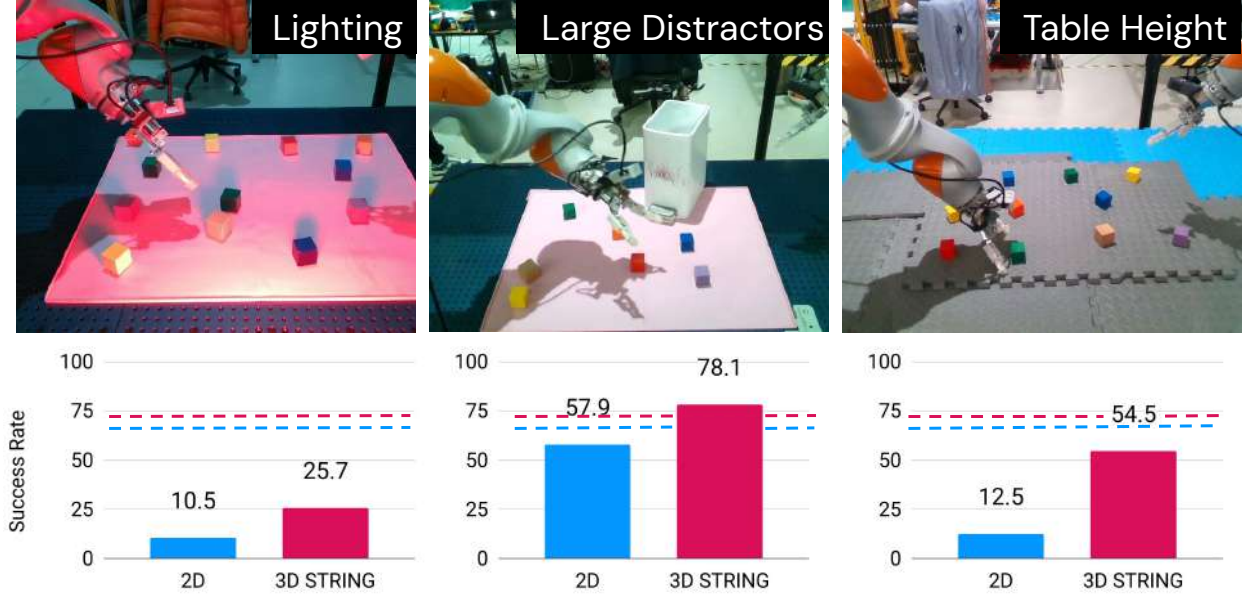


Figure 6 | Comparison of 2D baseline with 3D STRING on out-of-distribution scenarios for real-world Kuka robot evaluations.

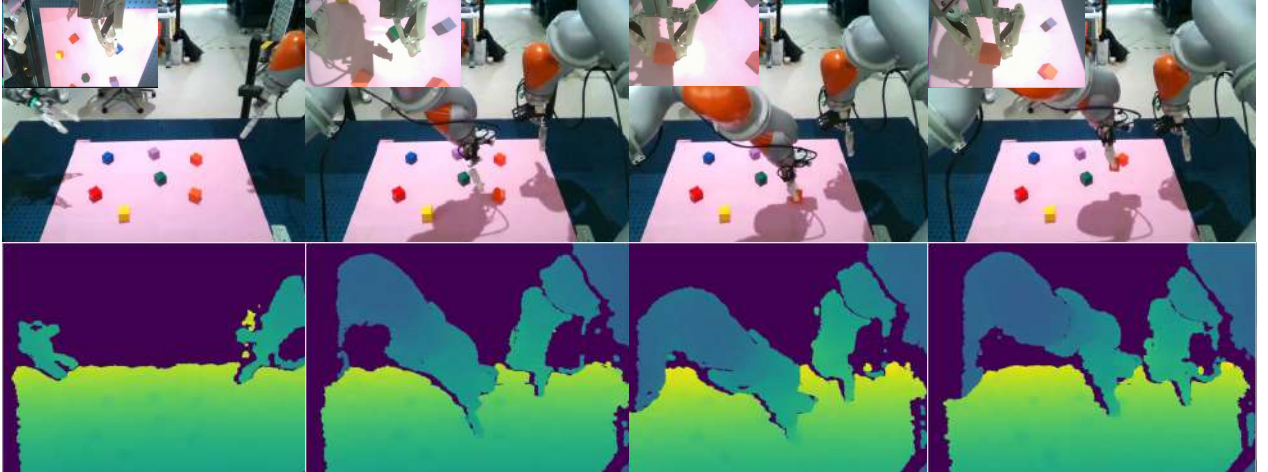


Figure 7 | Rollout frames from a successful policy evaluation in the real world (top: RGB, bottom: depth). Best viewed zoomed-in.

From 2D to 3D with STRING: We have already demonstrated (see the normal map approach from Section 4.4.2) that just adding a depth signal does not necessarily improve performance. STRING does so and exhibits another feature that other approaches (e.g. adding depth as extra channel) do not: it can be trained from a regular 2D pre-trained checkpoint. This is the case since STRING incorporates depth by using it to modulate a regular attention matrix, effectively disentangling 3D specific parameters (defining the modulation) from the core 2D backbone. All training runs in Section 4.4 started from the pre-trained 2D backbones.

5. Conclusion

We introduced a new class of translationally invariant position encodings (PEs) for Transformers, called STRING. STRING is the most general of all translation-invariant PE methods using matrix multiplications (under weak smoothness assumptions) and contains the prominent class of RoPE methods as its special instantiation. We proposed to apply STRING in robotics for 2D and 3D modeling and provided its extensive empirical verification over a range of tasks, from standard classification and retrieval, through object localization, to diffusion robotic policies conditioned on Vision Transformers. In all these experiments, we showed consistent gains over RoPE, as well as baselines applying regular absolute position encodings.

6. Impact Statement

The goal of this work is to contribute to the advancement of the machine learning field which may result in potential societal consequences. We acknowledge these potential risks, especially in downstream use-cases of advanced machine learning techniques and advocate for careful consideration of ethical implications in the development and deployment of these techniques. Additionally, as it is the case for all papers discussing training Transformer architectures, the corresponding carbon footprint needs to be taken into account. STRING plays a positive role here since it reduces computational costs by providing ways of fine-tuning already pre-trained architectures with a negligible set of extra trainable parameters.

References

- [1] Federico Barbero, Alex Vitvitskyi, Christos Perivolaropoulos, Razvan Pascanu, and Petar Veličković. Round and round we go! what makes rotary positional encodings useful? *arXiv preprint arXiv:2410.06205*, 2024.
- [2] Lucas Beyer, Andreas Steiner, André Susano Pinto, Alexander Kolesnikov, Xiao Wang, Daniel Salz, Maxim Neumann, Ibrahim Alabdulmohsin, Michael Tschannen, Emanuele Bugliarello, Thomas Unterthiner, Daniel Keysers, Skanda Koppula, Fangyu Liu, Adam Grycner, Alexey Gritsenko, Neil Houlsby, Manoj Kumar, Keran Rong, Julian Eisenschlos, Rishabh Kabra, Matthias Bauer, Matko Bošnjak, Xi Chen, Matthias Minderer, Paul Voigtlaender, Ioana Bica, Ivana Balazevic, Joan Puigcerver, Pinelopi Papalampidi, Olivier Henaff, Xi Xiong, Radu Soricut, Jeremiah Harmsen, and Xiaohua Zhai. PaliGemma: A versatile 3B VLM for transfer. *arXiv preprint arXiv:2407.07726*, 2024.
- [3] David Bindel, James Demmel, William Kahan, and Osni Marques. On computing givens rotations reliably and efficiently. *ACM Trans. Math. Softw.*, 28(2):206–238, 2002. doi: 10.1145/567806.567809. URL <https://doi.org/10.1145/567806.567809>.
- [4] Berk Calli, Aaron Walsman, Arjun Singh, Siddhartha Srinivasa, Pieter Abbeel, and Aaron M Dollar. Benchmarking in manipulation research: The ycb object and model set and benchmarking protocols. *arXiv preprint arXiv:1502.03143*, 2015.
- [5] Boyuan Chen, Zhuo Xu, Sean Kirmani, Brian Ichter, Danny Driess, Pete Florence, Dorsa Sadigh, Leonidas Guibas, and Fei Xia. Spatialvlm: Endowing vision-language models with spatial reasoning capabilities, 2024. URL <https://arxiv.org/abs/2401.12168>.
- [6] Pu-Chin Chen, Henry Tsai, Srinadh Bhojanapalli, Hyung Won Chung, Yin-Wen Chang, and Chun-Sung Ferng. A simple and effective positional encoding for transformers. In Marie-

- Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih, editors, *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*, pages 2974–2988. Association for Computational Linguistics, 2021. doi: 10.18653/V1/2021.EMNLP-MAIN.236. URL <https://doi.org/10.18653/v1/2021.emnlp-main.236>.
- [7] Xi Chen, Xiao Wang, Soravit Changpinyo, AJ Piergiovanni, Piotr Padlewski, Daniel Salz, Sebastian Goodman, Adam Grycner, Basil Mustafa, Lucas Beyer, Alexander Kolesnikov, Joan Puigcerver, Nan Ding, Keran Rong, Hassan Akbari, Gaurav Mishra, Linting Xue, Ashish Thapliyal, James Bradbury, Weicheng Kuo, Mojtaba Seyedhosseini, Chao Jia, Burcu Karagol Ayan, Carlos Riquelme, Andreas Steiner, Anelia Angelova, Xiaohua Zhai, Neil Houlsby, and Radu Soricut. Pali: A jointly-scaled multilingual language-image model, 2023. URL <https://arxiv.org/abs/2209.06794>.
- [8] Ta-Chung Chi, Ting-Han Fan, Peter J. Ramadge, and Alexander Rudnicky. KERPLE: kernelized relative positional embedding for length extrapolation. In Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022.
- [9] Ta-Chung Chi, Ting-Han Fan, Alexander Rudnicky, and Peter J. Ramadge. Dissecting transformer length extrapolation via the lens of receptive field analysis. In Anna Rogers, Jordan L. Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2023, Toronto, Canada, July 9-14, 2023*, pages 13522–13537. Association for Computational Linguistics, 2023. doi: 10.18653/V1/2023.ACL-LONG.756. URL <https://doi.org/10.18653/v1/2023.acl-long.756>.
- [10] Krzysztof Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. Rethinking attention with performers. *arXiv preprint arXiv:2009.14794*, 2020.
- [11] Krzysztof Choromanski, Han Lin, Haoxian Chen, Tianyi Zhang, Arijit Sehanobish, Valerii Likhoshesterov, Jack Parker-Holder, Tamas Sarlos, Adrian Weller, and Thomas Weingarten. From block-toeplitz matrices to differential equations on graphs: towards a general theory for scalable masked transformers. In *International Conference on Machine Learning*, pages 3962–3983. PMLR, 2022.
- [12] Jasmine Collins, Shubham Goel, Kenan Deng, Achleshwar Luthra, Leon Xu, Erhan Gundogdu, Xi Zhang, Tomas F Yago Vicente, Thomas Dideriksen, Himanshu Arora, Matthieu Guillaumin, and Jitendra Malik. Abo: Dataset and benchmarks for real-world 3d object understanding. *CVPR*, 2022.
- [13] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009. doi: 10.1109/CVPR.2009.5206848.
- [14] Fasma Diele, Luciano Lopez, and R. Peluso. The cayley transform in the numerical solution of unitary differential systems. *Adv. Comput. Math.*, 8(4):317–334, 1998. doi: 10.1023/A:1018908700358. URL <https://doi.org/10.1023/A:1018908700358>.
- [15] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *ICLR*, 2021.

- [16] Laura Downs, Anthony Francis, Nate Koenig, Brandon Kinman, Ryan Hickman, Krista Reymann, Thomas B McHugh, and Vincent Vanhoucke. Google scanned objects: A high-quality dataset of 3d scanned household items. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 2553–2560. IEEE, 2022.
- [17] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [18] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, et al. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024.
- [19] Agrim Gupta, Piotr Dollar, and Ross Girshick. Lvis: A dataset for large vocabulary instance segmentation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 5356–5364, 2019.
- [20] Brian C Hall. *Lie groups, Lie algebras, and representations*. Springer, 2013.
- [21] Dan Hendrycks and Kevin Gimpel. Bridging nonlinearities and stochastic regularizers with gaussian error linear units. *CoRR*, abs/1606.08415, 2016. URL <http://arxiv.org/abs/1606.08415>.
- [22] Byeongho Heo, Song Park, Dongyoon Han, and Sangdoo Yun. Rotary position embedding for vision transformer. In *European Conference on Computer Vision*, pages 289–305. Springer, 2025.
- [23] Christoph Hertzberg, René Wagner, Udo Frese, and Lutz Schröder. Integrating generic sensor fusion algorithms with sound state representations through encapsulation of manifolds. *CoRR*, abs/1107.1119, 2011. URL <http://arxiv.org/abs/1107.1119>.
- [24] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pages 5156–5165. PMLR, 2020.
- [25] Amirhossein Kazemnejad, Inkit Padhi, Karthikeyan Natesan Ramamurthy, Payel Das, and Siva Reddy. The impact of positional encoding on length generalization in transformers. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine, editors, *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023.
- [26] Shun Kiyono, Sosuke Kobayashi, Jun Suzuki, and Kentaro Inui. SHAPE: Shifted absolute position embedding for transformers. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih, editors, *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3309–3321, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.emnlp-main.266. URL <https://aclanthology.org/2021.emnlp-main.266/>.
- [27] H.W. Kuhn. The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.
- [28] Shanda Li, Chong You, Guru Guruganesh, Joshua Ainslie, Santiago Ontañón, Manzil Zaheer, Sumit Sanghai, Yiming Yang, Sanjiv Kumar, and Srinadh Bhojanapalli. Functional interpolation for relative positions improves long context transformers. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=rR03qFesqk>.

- [29] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6-12, 2014, Proceedings, Part V 13*, pages 740–755. Springer, 2014.
- [30] Xuanqing Liu, Hsiang-Fu Yu, Inderjit S. Dhillon, and Cho-Jui Hsieh. Learning to encode position for transformer with continuous dynamical model. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, volume 119 of *Proceedings of Machine Learning Research*, pages 6327–6335. PMLR, 2020. URL <http://proceedings.mlr.press/v119/liu20n.html>.
- [31] Shengjie Luo, Shanda Li, Tianle Cai, Di He, Dinglan Peng, Shuxin Zheng, Guolin Ke, Liwei Wang, and Tie-Yan Liu. Stable, fast and accurate: Kernelized attention with relative positional encoding. *Advances in Neural Information Processing Systems*, 34:22795–22807, 2021.
- [32] Austin Stone Maxim Neumann Dirk Weissenborn Alexey Dosovitskiy Aravindh Mahendran Anurag Arnab Mostafa Dehghani Zhuoran Shen Xiao Wang Xiaohua Zhai Thomas Kipf Neil Houlsby Matthias Minderer, Alexey Gritsenko. Simple open-vocabulary object detection with vision transformers. *ECCV*, 2022.
- [33] Sophie Ostmeier, Brian Axelrod, Michael E Moseley, Akshay Chaudhari, and Curtis Langlotz. Liere: Generalizing rotary position encodings. *arXiv preprint arXiv:2406.10322*, 2024.
- [34] Ofir Press, Noah A. Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022. URL <https://openreview.net/forum?id=R8sQPpGCv0>.
- [35] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR, 2021.
- [36] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.*, 21:140:1–140:67, 2020. URL <https://jmlr.org/papers/v21/20-074.html>.
- [37] Isaac Reid, Kumar Avinava Dubey, Deepali Jain, Will Whitney, Amr Ahmed, Joshua Ainslie, Alex Bewley, Mithun Jacob, Aranyak Mehta, David Rendleman, et al. Linear transformer topological masking with graph random features. *arXiv preprint arXiv:2410.03462*, 2024.
- [38] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.
- [39] Jacob Rubinstein, Joseph Segman, and Yehoshua Zeevi. Recognition of distorted patterns by invariance kernels. *Pattern Recognition*, 24(10):959–967, 1991.
- [40] Joseph Segman, Jacob Rubinstein, and Yehoshua Y Zeevi. The canonical coordinates method for pattern deformation: Theoretical and computational considerations. *IEEE Transactions on Pattern Analysis & Machine Intelligence*, 14(12):1171–1183, 1992.

- [41] Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. Self-attention with relative position representations. *arXiv preprint arXiv:1803.02155*, 2018.
- [42] Sumeet Singh, Stephen Tu, and Vikas Sindhwani. Revisiting energy based models as policies: Ranking noise contrastive estimation and interpolating energy models. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL <https://openreview.net/forum?id=JmKAYb7I00>.
- [43] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- [44] Kai Sheng Tai, Peter Bailis, and Gregory Valiant. Equivariant transformer networks. In *International Conference on Machine Learning*, pages 6086–6095. PMLR, 2019.
- [45] Georgios Tzifas and Hamidreza Kasaei. Early or late fusion matters: Efficient rgb-d fusion in vision transformers for 3d object recognition. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 9558–9565, 2023.
- [46] J. Divya Udayan, Veerababu Addanki, Sathvik Durgapu, Dhanvanth Reddy Yerramreddy, and Dorasanaiah Kolla. Forward kinematics simulation of KUKA KR5 arc robot with robo analyzer. In *Proceedings of the 2023 Fifteenth International Conference on Contemporary Computing, IC3-2023, Noida, India, August 3-5, 2023*, pages 294–299. ACM, 2023. doi: 10.1145/3607947.3608006. URL <https://doi.org/10.1145/3607947.3608006>.
- [47] Unity Technologies. Unity, 2023. URL <https://unity.com/>. Game development platform.
- [48] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pages 5998–6008, 2017.
- [49] Benyou Wang, Lifeng Shang, Christina Lioma, Xin Jiang, Hao Yang, Qun Liu, and Jakob Grue Simonsen. On position embeddings in BERT. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. URL <https://openreview.net/forum?id=onxoVA9FxMw>.
- [50] Wenhan Xiong, Jingyu Liu, Igor Molybog, Hejia Zhang, Prajjwal Bhargava, Rui Hou, Louis Martin, Rashi Rungta, Karthik Abinav Sankararaman, Barlas Oguz, et al. Effective long-context scaling of foundation models. *arXiv preprint arXiv:2309.16039*, 2023.
- [51] Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything v2, 2024. URL <https://arxiv.org/abs/2406.09414>.
- [52] Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. *arXiv preprint arXiv:2303.15343*, 2023.
- [53] Liang Zhao, Xiaocheng Feng, Xiachong Feng, Bing Qin, and Ting Liu. Length extrapolation of transformers: A survey from the perspective of position encoding. *CoRR*, abs/2312.17044, 2023. doi: 10.48550/ARXIV.2312.17044.
- [54] Tony Z Zhao, Jonathan Tompson, Danny Driess, Pete Florence, Kamyar Ghasemipour, Chelsea Finn, and Ayzaan Wahid. Aloha unleashed: A simple recipe for robot dexterity. *arXiv preprint arXiv:2410.13126*, 2024.

Contributions

Connor Schenck worked on the pipeline to filter the full WebLI dataset of 10 billion images down to approximately 60 million images and run the monodepth models on the RGB images to create depth images. Connor also created the codebase to train models to predict 3D bounding boxes for queried objects in a scene and generated the results for 3D detection in section 4.2.2.

Isaac Reid proposed to parameterise PEs as exponentiated learnable generators, proved Theorems 3.2-3.4, wrote Secs 1-3 of the manuscript.

Mithun George Jacob proposed and implemented Cayley-STRING. He evaluated it on WebLI-3D, ALOHA simulation, ImageNet and Places365. He also contributed to the creation of WebLI-3D, the integration of STRING into ALOHA Unleashed and the preparation of the manuscript and website.

Alex Bewley carried out the OWL-ViT 2D experiments with RoPE and STRING variants. Alex also reviewed and contributed to the design of the 3D bounding box detection model used in the experiment and helped prepare the manuscript.

Joshua Ainslie proposed investigating RoPE for robot vision with RGB/RGB-D inputs and developed the library for learnable 2D/3D RoPE in ViT/ViT-D models. He set up early experiments for WebLI-3D, ImageNet, Places365, and OWL-ViT indicating promising headroom for improved position encodings.

David Rendleman contributed to WebLI-3D dataset creation, ViT training infrastructure, and evaluation of trained models in simulated ALOHA.

Deepali Jain helped set up the sim dataset for 3D detection. Ran training experiments for on-robot manipulation tasks and compared several RELM-based vision encoders for generative policies.

Mohit Sharma co-developed training, evaluation and infrastructure for Kuka real-world experiments. Setup and ran training experiments to evaluate 3D encoders on real-world tasks. Helped write the real-world Kuka evaluations section of the paper.

Avinava Dubey developed RoPE for ViT/ViT-D and experimented with WebLI-3D, ImageNet and Places365. Helped with writing the paper.

Ayzaan Wahid provided code and support for running ALOHA experiments in sim and real: train codebase, support with sim experiments, support repo for real deployment.

Sumeet Singh integration support for Diffusion- ϕ [42], multi-system and assist training.

René Wagner advised on orientation representations, proposed switching from quaternions to the two-column orientation representation used for 3D detection results, and helped prepare the manuscript.

Tianli Ding advised on datasets selection and monocular depth estimation, conducted ALOHA real-robot experiments.

Chuyuan Fu provided code and support for generating and using ALOHA sim dataset.

Arun Kumar Byravan provided the sim datasets for 3D detection, Colab for loading the results and support in working with the datasets.

Jake Varley provided code and support for running Kuka experiments.

Alexey Gritsenko advised on OWL-ViT with RoPE and 3D detection experiments.

Matthias Minderer advised on the use of the Scenic codebase for OWL-ViT experiments and the inner workings of OWL-ViT architecture used in both the 2D and 3D experiments.

Dmitry Kalashnikov bimanual Kuka data infra development and support. Red-teaming infra and research contributions.

Jonathan Tompson advised on ALOHA sim and real experiments.

Vikas Sindhwani led red-teaming and out-of-distribution data collection efforts on biarm Kuka cells; supported 3d + Diffusion- ϕ effort.

Krzysztof Choromanski overall lead of the project (managing project and Team). Co-proposed Lie-algebra approach to extend RoPE embeddings. Developed and implemented Circulant-STRING and proved Theorem 3.5. Proposed Extension 1 and co-proposed Extension 2. Prepared depth modeling backbone for Diffusion- ϕ experiments and trained first depth Diffusion- ϕ policies conditioned on STRING for robotics manipulation with KUKA arms. Helped write the paper and prepare the website.

A. Proofs

A.1. Proof of Theorem 3.2

To begin, we provide a proof of Theorem 3.2: that STRING is the most general form of transformation that respects the group-like property

$$\mathbf{R}(\mathbf{r}_i)^\top \mathbf{R}(\mathbf{r}_j) = \mathbf{R}(\mathbf{r}_j - \mathbf{r}_i), \quad (15)$$

supposing that $\mathbf{R}(\mathbf{0}) = \mathbf{I}_d$ and $\mathbf{R}(\mathbf{r})$ is continuously differentiable with respect to $\mathbf{r}$. Note that this is a sufficient, but not necessary, assumption for translational invariance.

Proof. Recall STRING is applied as

$$\mathbf{q}_i \rightarrow \mathbf{R}(\mathbf{r}_i) \mathbf{q}_i \quad (16)$$

with $\mathbf{R}(\cdot) : \mathbb{R}^{d_c} \rightarrow \mathbb{R}^{d \times d}$, so that $\mathbf{R}(\mathbf{r}_i) \in \mathbb{R}^{d \times d}$. Then we require that

$$\mathbf{q}_i^\top \mathbf{k}_j \rightarrow \mathbf{q}_i^\top \mathbf{R}(\mathbf{r}_i)^\top \mathbf{R}(\mathbf{r}_j) \mathbf{k}_j. \quad (17)$$

Recall that $\mathbf{R}(\mathbf{0}) = \mathbf{I}_d$, the d -dimensional identity, so that the logit of a query and key at the same position ($\mathbf{r}_i = \mathbf{r}_j$) will be unmodified by the positional encoding. Then clearly $\mathbf{R}(\mathbf{r}_i) \in \text{O}(d)$, the orthogonal group in dimension d . For compatibility with gradient-based optimisers (a chief concern in the machine learning setting), it is convenient to specialise to the connected component (normal subgroup) of $\text{O}(d)$ containing the identity matrix: that is, the special orthogonal group $\text{SO}(d)$.⁶ This means that $\det(\mathbf{R}(\mathbf{r}_i)) = 1$. These transformations are the d -dimensional rotations.

Since $\mathbf{R}(\mathbf{r}_i) \in \text{SO}(d)$, the rotation can be written using its Lie group,

$$\mathbf{R}(\mathbf{r}_i) = \exp(\mathbf{L}(\mathbf{r}_i)) \quad (18)$$

where the matrix $\mathbf{L}(\mathbf{r}_i)$ is antisymmetric [20]. $\mathbf{L}(\mathbf{r}_i)$ is called the ‘generator’, representing an infinitesimal rotation. Here, $\exp(\cdot)$ denotes the *matrix* exponential (not to be confused with the element-wise exponential of a matrix, as appears e.g. in softmax). Setting $\mathbf{r}_j = \mathbf{0}$ in Equation (15), it is clear that $\mathbf{L}(-\mathbf{r}_i) = -\mathbf{L}(\mathbf{r}_i)$. We then require that

$$\begin{aligned} \exp(\mathbf{L}(\mathbf{r}_i)) \exp(\mathbf{L}(\mathbf{r}_j)) &= \exp(\mathbf{L}(\mathbf{r}_i + \mathbf{r}_j)) \\ &= \exp(\mathbf{L}(\mathbf{r}_j)) \exp(\mathbf{L}(\mathbf{r}_i)). \end{aligned} \quad (19)$$

Clearly $\mathbf{L}(\mathbf{r}_i)$ and $\mathbf{L}(\mathbf{r}_j)$ must commute for all choices of coordinate vector $(\mathbf{r}_i, \mathbf{r}_j)$. Therefore, we need

$$\mathbf{L}(\mathbf{r}_i + \mathbf{r}_j) = \mathbf{L}(\mathbf{r}_i) + \mathbf{L}(\mathbf{r}_j), \quad (20)$$

so $\mathbf{L}(\cdot)$ is linear in its arguments. That is, $\mathbf{L}(\cdot)$ is a linear map from the set of d_c -dimensional vectors to a set of commuting antisymmetric matrices. We can write

$$\mathbf{L}(\mathbf{r}_i) = \sum_{k=1}^{d_c} \mathbf{L}_k [\mathbf{r}_i]_k, \quad (21)$$

with $\{\mathbf{L}_k\}_{k=1}^{d_c} \subset \mathbb{R}^{d \times d}$ a set of *commuting antisymmetric generators* and $[\mathbf{r}_i]_k$ the k -th entry of coordinate vector $\mathbf{r}_i$. This completes the proof. $\square$

⁶This means that you can optimise the position encoding transformations on the same manifold. You could in principle also incorporate reflections so that $\det(\mathbf{R}(\mathbf{r}_i)) = -1$, but this seems unlikely to significantly improve performance.

A.2. Proof of Theorem 3.3

Now, we prove that generators of the form $\mathbf{L}_k = \sum_{p=1}^{d/2} (\delta_{2p,2p-1} - \delta_{2p-1,2p})\theta_p$ recover RoPE, as described in Theorem 3.3.

Proof. Let us initially consider the case $d_c = 1$, so that the token coordinate $\mathbf{r} = r \in \mathbb{R}$ and we learn a single generator. Recall that powers of a block diagonal matrix will remain block diagonal. Each block of the generator $\mathbf{L}_k$ is of the form

$$\begin{bmatrix} 0 & \theta \\ -\theta & 0 \end{bmatrix}. \quad (22)$$

Then note that

$$\begin{bmatrix} 0 & \theta \\ -\theta & 0 \end{bmatrix}^2 = \begin{bmatrix} -\theta^2 & 0 \\ 0 & -\theta^2 \end{bmatrix}. \quad (23)$$

It follows that

$$\begin{bmatrix} 0 & \theta \\ -\theta & 0 \end{bmatrix}^n = \begin{cases} \theta^n (-1)^{n/2} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} & \text{if } n \text{ is even,} \\ \theta^n (-1)^{(n-1)/2} \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} & \text{if } n \text{ is odd.} \end{cases} \quad (24)$$

Combining and inspecting the Taylor expansions,

$$\exp \begin{bmatrix} 0 & \theta \\ -\theta & 0 \end{bmatrix} = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix}, \quad (25)$$

which is clearly a rotation matrix – a well-known result. This holds for all the $d/2$ blocks, each of which exponentiates to give a 2×2 rotation at a different frequency. Therefore,

$$\exp(\mathbf{L}r) = \text{RoPE}(r), \quad (26)$$

showing that with this particular generator STRING is RoPE.

Now suppose that $d_c > 1$, so $\mathbf{L}(\mathbf{r}_i) = \sum_{k=1}^{d_c} \mathbf{L}_k [\mathbf{r}_i]_k$. In our special case, each generator is of the form $\mathbf{L}_k = \sum_{p=1}^{d/2} (\delta_{2p,2p-1} - \delta_{2p-1,2p})\theta_p$, where $\{\theta_p\}_{p=1}^{d/2}$ can differ for different k (notationally suppressed for compactness). Observing that

$$\begin{aligned} \begin{bmatrix} 0 & \alpha \\ -\alpha & 0 \end{bmatrix} \begin{bmatrix} 0 & \beta \\ -\beta & 0 \end{bmatrix} &= \begin{bmatrix} -\alpha\beta & 0 \\ 0 & -\alpha\beta \end{bmatrix} \\ &= \begin{bmatrix} 0 & \beta \\ -\beta & 0 \end{bmatrix} \begin{bmatrix} 0 & \alpha \\ -\alpha & 0 \end{bmatrix}, \end{aligned} \quad (27)$$

different $\mathbf{L}_k [\mathbf{r}_i]_k$ commute. Then we have that

$$\begin{aligned} \exp \left(\sum_{k=1}^{d_c} \mathbf{L}_k [\mathbf{r}_i]_k \right) &= \prod_{k=1}^{d_c} \exp (\mathbf{L}_k [\mathbf{r}_i]_k) \\ &= \prod_{k=1}^{d_c} \text{RoPE}([\mathbf{r}_i]_k) = \text{RoPE}(\mathbf{r}), \end{aligned} \quad (28)$$

where we used the definition for multidimensional RoPE from Equation (5). This completes the proof. $\square$

A.3. Proof of Theorem 3.4

Next, we prove that STRING can always be rewritten as RoPE in a different basis.

Proof. As usual, we begin with $d_c = 1$. Then our task is to show that a matrix $\mathbf{R}(r) = \exp(\mathbf{L}r)$, with $\mathbf{L} \in \mathbb{R}^{d \times d}$ an antisymmetric matrix and $r \in \mathbb{R}$, can be rewritten as RoPE with a change of basis.

Begin by noting that $\mathbf{R}(r) \in \text{SO}(d)$; it is a special orthogonal matrix. It is orthogonal since $\mathbf{R}(r)^\top \mathbf{R}(r) = \exp(\mathbf{L}r)^\top \exp(\mathbf{L}r) = \exp(-\mathbf{L}r) \exp(\mathbf{L}r) = \mathbf{I}_d$, and its determinant is 1 since it is continuously connected to the identity (which occurs at $r = 0$). Consider an eigenvector $\mathbf{v} \in \mathbb{C}^d$, with eigenvalue $\lambda \in \mathbb{C}$. Since $\mathbf{R}^\top \mathbf{R} = \mathbf{I}_d$, $|\lambda| = 1$ so $\lambda = e^{i\theta}$. Taking the complex conjugate of $\mathbf{R}\mathbf{v} = \lambda\mathbf{v}$, we have $\mathbf{R}\bar{\mathbf{v}} = \lambda^* \bar{\mathbf{v}}$, where $\lambda^* = e^{-i\theta}$ and $\bar{\mathbf{v}}$ is the complex conjugate of $\mathbf{v}$. We used that $\mathbf{R}$ is real. Therefore, the eigenvalues appear in conjugate pairs for conjugate eigenvectors.

Let $\mathbf{v} = \mathbf{u} + i\mathbf{w}$ and $\bar{\mathbf{v}} = \mathbf{u} - i\mathbf{w}$ with $\mathbf{u}, \mathbf{w} \in \mathbb{R}^d$ real vectors. Inserting into the eigenvector equation, $\mathbf{R}(\mathbf{u} + i\mathbf{w}) = (\cos(\theta) + i\sin(\theta))(\mathbf{u} + i\mathbf{w})$. Equating the real and imaginary parts, $\mathbf{R}\mathbf{u} = \cos(\theta)\mathbf{u} - \sin(\theta)\mathbf{w}$ and $\mathbf{R}\mathbf{w} = \sin(\theta)\mathbf{u} + \cos(\theta)\mathbf{w}$. This corresponds exactly to 2-dimensional rotation in the $(\mathbf{u}, \mathbf{w})$ plane. By normalisation of (complex) $\mathbf{v}$, $|\mathbf{u}|^2 + |\mathbf{w}|^2 = 1$. But since λ and λ^* differ, their corresponding eigenvectors are orthogonal under the Hermitian inner product, so we also have that $\bar{\mathbf{v}}^\top \mathbf{v} = |\mathbf{u}|^2 - |\mathbf{w}|^2 + 2i\mathbf{u}^\top \mathbf{w} = 0$. So $\mathbf{u}^\top \mathbf{w} = 0$, whereupon $\mathbf{u}$ and $\mathbf{w}$ are orthogonal vectors in $\mathbb{R}^d$. Of course, from basic linear algebra the complex eigenvectors corresponding to *different* θ will also be orthogonal in $\mathbb{C}^d$, or in the case of degenerate θ one can choose an orthogonal basis for the corresponding subspace using e.g. the Gram-Schmidt process. It is easy to show that the corresponding real vectors $(\mathbf{u}_i, \mathbf{w}_i)$ will therefore be orthogonal for different i , i.e. $\mathbf{u}_i^\top \mathbf{u}_j = \delta_{i,j} = \mathbf{w}_i^\top \mathbf{w}_j$. To summarise: the real and imaginary parts of the d orthogonal (complex) eigenvectors of $\mathbf{R}$ in $\mathbb{C}^d$ correspond to d orthogonal (real) vectors in $\mathbb{R}^d$, organised in $\frac{d}{2}$ planes in each of which $\mathbf{R}$ acts as a 2-dimensional rotation, at a frequency that depends on the corresponding eigenvalue. These real vectors $\{\mathbf{u}_i, \mathbf{w}_i\}_{i=1}^{d/2}$ can be aggregated as the columns an orthogonal matrix $\mathbf{P}$, taking

$$\mathbf{P} := \begin{bmatrix} \uparrow & \uparrow & \uparrow & \uparrow & \cdots \\ \mathbf{u}_1 & \mathbf{w}_1 & \mathbf{u}_2 & \mathbf{w}_2 & \cdots \\ \downarrow & \downarrow & \downarrow & \downarrow & \cdots \end{bmatrix} \in \mathbb{R}^{d \times d} \quad (29)$$

whereupon

$$\mathbf{R}(r) = \mathbf{P} \text{RoPE}(r) \mathbf{P}^\top. \quad (30)$$

Note especially that the change of basis matrix $\mathbf{P}$ is independent of r , because $\mathbf{P}$ determines the axes of rotation whereas the (complex) eigenvalues depend on the amount of rotation. This is obvious from the definition $\mathbf{R}(r) = \exp(\mathbf{L}r)$; the matrix $\mathbf{P}$ needs to (block) diagonalise $\mathbf{L}$, then this is sufficient to (block) diagonalise $\mathbf{R}$, and the matrix $\mathbf{L}$ is independent of $\mathbf{r}$.

Now suppose that $d_c > 1$. Recall that we have

$$\mathbf{R}(\mathbf{r}) = \exp \sum_{k=1}^{d_c} \mathbf{L}_k \mathbf{r}_k = \prod_{k=1}^{d_c} \exp \mathbf{L}_k \mathbf{r}_k = \prod_{k=1}^{d_c} \mathbf{R}([\mathbf{r}]_k), \quad (31)$$

where we used the fact that the generators $\{\mathbf{L}_k\}_{k=1}^{d_c}$ commute (Definition 3.1). $\{\mathbf{R}(\mathbf{r}_k)\}_{k=1}^{d_c}$ must also commute so are simultaneously diagonalisable, whereupon

$$\begin{aligned} \mathbf{R}(\mathbf{r}) &= \prod_{k=1}^{d_c} \mathbf{P} \text{RoPE}([\mathbf{r}]_k) \mathbf{P}^\top = \mathbf{P} \left(\prod_{k=1}^{d_c} \text{RoPE}([\mathbf{r}]_k) \right) \mathbf{P}^\top \\ &= \mathbf{P} \text{RoPE}(\mathbf{r}) \mathbf{P}^\top. \end{aligned} \quad (32)$$

This concludes the proof. $\square$

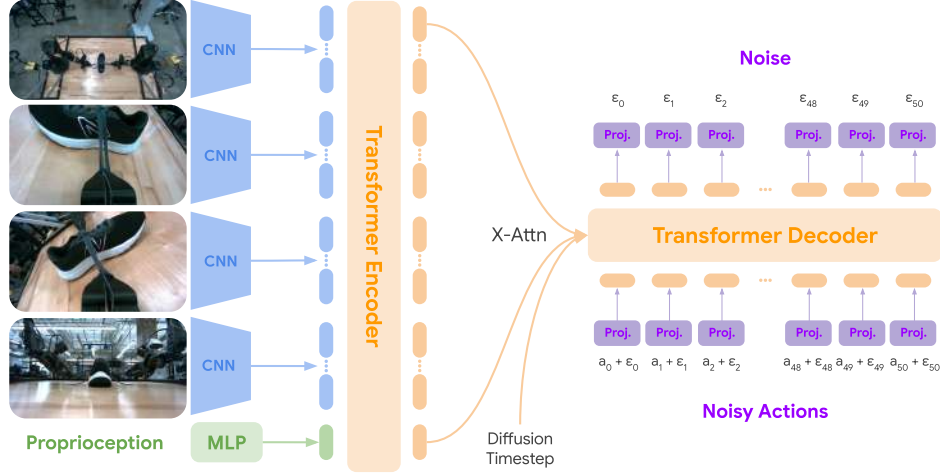


Figure 8 | ALOHA Unleashed Architecture [54].

A.4. Proof of Theorem 3.5

Proof. Note that it suffices to show that the computation of $\mathbf{p} = \exp(\mathbf{C} - \mathbf{C}^\top)\mathbf{z}$ can be computed in time $O(d \log(d))$ and memory $O(d)$ for a circulant matrix $\mathbf{C}$, defined by its first row $\mathbf{c}$. We will leverage the fact that every circulant matrix $\mathbf{C}$ can be factorized as follows:

$$\mathbf{C} = \mathbf{DFT} \times \text{diag}(\mathbf{DFT}\mathbf{c}) \times \mathbf{DFT}^{-1}, \quad (33)$$

for the Discrete Fourier Transform matrix $\mathbf{DFT}$, and where $\text{diag}(\mathbf{v})$ stands for the diagonal matrix with the main diagonal given by $\mathbf{v}$. Therefore we have the following:

$$\mathbf{p} = \exp(\mathbf{DFT}(\text{diag}(\mathbf{DFT}\mathbf{u})))\mathbf{DFT}^{-1}\mathbf{z}, \quad (34)$$

where $\mathbf{u} = \mathbf{c} - \mathbf{t}$ and $\mathbf{t}$ stands for the first column of $\mathbf{C}$. Here we leverage the fact that the transpose of the circulant matrix is also circulant. Therefore, by leveraging Taylor series formula for $\exp$, we get:

$$\begin{aligned} \mathbf{p} &= \mathbf{DFT} \exp(\text{diag}(\mathbf{DFT}\mathbf{u}))\mathbf{DFT}^{-1}\mathbf{z} = \\ &\quad \mathbf{DFT} \text{diag}(\exp(\mathbf{DFT}\mathbf{u}))\mathbf{DFT}^{-1}\mathbf{z}, \end{aligned} \quad (35)$$

where $\exp$ in the last formula is computed element-wise. Note that matrix-vector product with diagonal matrices can be trivially conducted in linear time. Thus the calculation of $\mathbf{p}$ can be conducted in $O(d)$ memory and $O(d \log(d))$ time complexity via Fast Fourier Transform (FFT) (to multiply with $\mathbf{DFT}$ matrices) and inverse FFT (iFFT) (to multiply with matrices $\mathbf{DFT}^{-1}$). That completes the proof. $\square$

B. ALOHA Unleashed Simulation Tasks

ALOHA leader robots are teleoperated in ALOHA simulation for data collection using an ALOHA station, an Oculus VR headset and controllers [54]. The teleoperators are instructed to perform the following tasks using this setup. See Figure 9 and Figure 10 for simulation renders of the following 12 tasks.

- 1-3. {Bowl/Glass/Plate}OnRack: Place the item on the rack.
4. SingleInsertion: Use the left arm to grab the blue socket and the right arm to insert the block into the socket.

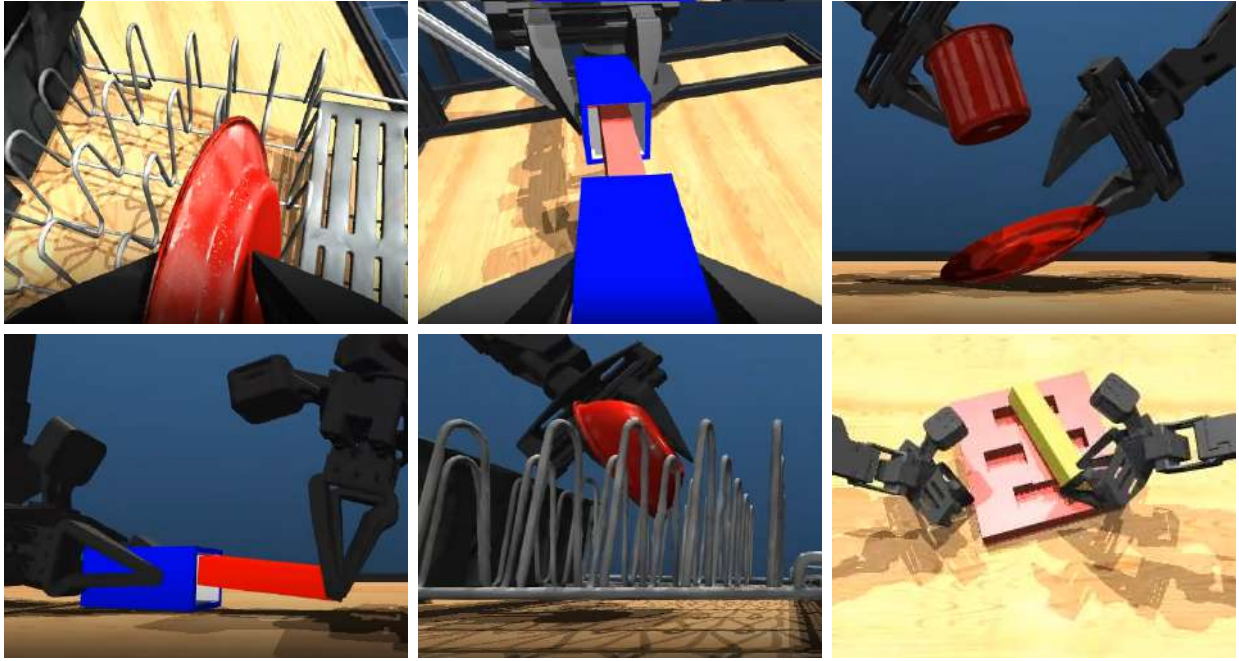


Figure 9 | Views from the wrist, overhead and table-level cameras in ALOHA sim. **Top to bottom, Left to right:** PlateOnRack, DoubleInsertion (insert peg into sockets on either end), MugOnPlate, SingleInsertion, BowlOnRack, and Functional Manipulation Benchmark-1 (FMB-1).

5. DoubleInsertion: After completing the SingleInsertion task, insert another block into the other end of the socket.
- 6-7. Functional Manipulation Benchmark (FMB) 1 and 2: Insert a yellow block into the recess of a red base.
8. FruitBowl: Place all the fruits in the bowl.
9. StorageBin: Place all the snack boxes in the storage bin.
- 10-11. HandOver{Banana/Pen}: Hand over the item and place it in the container.
12. MugOnPlate Place the mug on the plate.

MultiTask aggregates results of all of the above tasks.

C. Aloha Real Tasks

ALOHA-real models are first pre-trained with human-teleop data collected on 300 diverse tasks, which were crowd-sourced based on relevance with real-world scenarios as well as feasibility for the ALOHA robot. Further, the model is fine-tuned on the following 10 tasks.

1. open-jar-lid: open the glass jar lid, handover to other hand and put on the table
2. bowl-in-rack: put the bowl into the drying rack
3. cup-in-rack: put the cup into the drying rack
4. banana-handover: put banana in bowl with handover
5. open-drawer: open the drawer
6. remove-gears: remove the gears from the nist-board
7. fold-dress: fold the dress
8. stack-cups: stack the cups

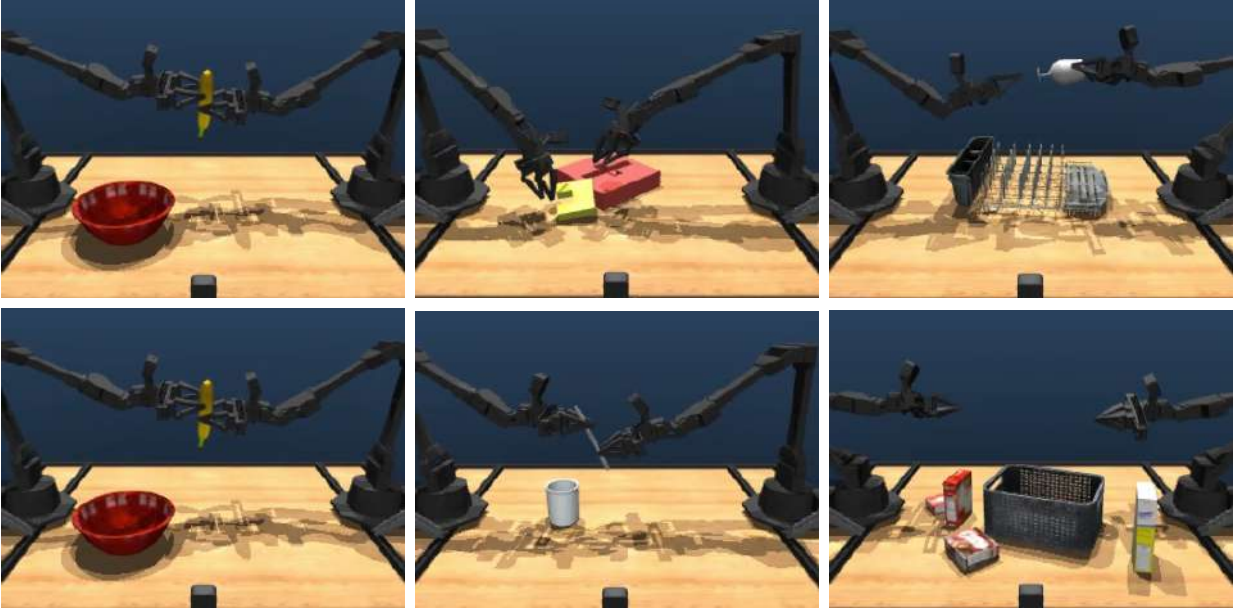


Figure 10 | More ALOHA Simulation Tasks. **Top to bottom, left to right:** FruitBowl, FMB-2, GlassOn-Rack, HandOverBanana, HandOverPen and StorageBin.

9. pen-handover: put pen in container with handover
10. take-phone-out: take phone out of purse

After fine-tuning, the model is then evaluated on the following 5 tasks: bowl-in-rack, banana-handover, bowl-in-rack, fold-dress, remove-gears. See: Fig. 11 for the visualizations of selected ALOHA real world tasks.

D. WebLI-3D Dataset

We lift the WebLI dataset [7], a dataset of 10 billion image-text pairs across a variety of languages, into 3D by pre-processing a 60-million image subset using Depth-Anything-V2 [51] for metric monodepth estimation. The dataset is filtered using the method described in [5] for images that the indoor-finetuned model performs poorly on (pictures with overlays, no visible groundplane, large outdoor scenes, optical illusions are removed).

E. Classification and Retrieval Experiment Details

Our experimental base model is ViT-B/16 [15], which has 12 layers, 768 model dimension, 3072 MLP size, 12 attention heads, and 16×16 patch size. All RoPE and STRING variants retain these same model hyperparameters. For vanilla RoPE, we use the common 10,000 max wavelength and simply split query/key dimensions between 2 or 3 axes in the 2D or 3D case, respectively. For RoPE-Mixed, we initialize with 100 max wavelength as suggested in Heo et al. [22], which we also adopt for STRING.

For other hyperparameters like learning rate, batch size, warm-up schedule, etc., we left these the same as the default values used for the base ViT-B/16 model. We use the Adam optimizer with $b1=0.9$ and $b2=0.999$ for all experiments. For STRING, we experimented with sharing parameters across

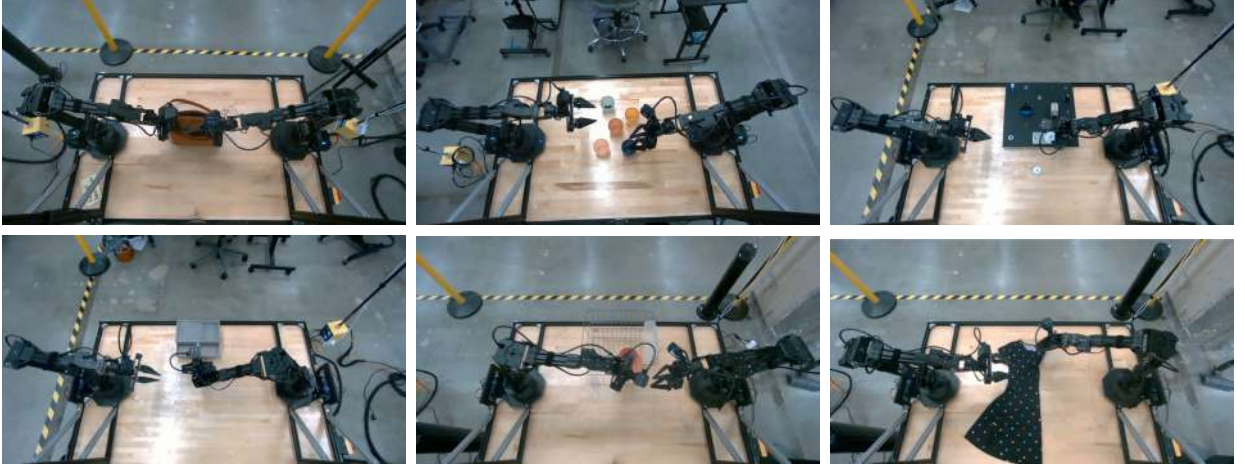


Figure 11 | Sampled ALOHA Real Tasks. **Top to bottom, left to right:** take-phone-out, stack-cup, remove-gear, open-drawer, bowl-in-rack and fold-dress.

attention heads rather than the default of learning them all separately, and we found this could yield slight gains in both efficiency and quality. Finally, for Circulant-STRING, we swept over block sizes in the set $\{4, 8, 16, 32, 64\}$ to find the optimal setting (often around 16).

E.1. ImageNet2012 and Places365 Classification

All experiments were trained from scratch, separately for ImageNet2012 or Places365. In both cases, we used 224×224 image resolution, batch size 4096, and trained for 300 epochs. Training used the cosine decay learning rate schedule with 0.001 base learning rate and 10,000 warm-up steps. For ImageNet2012 there were a total of about 94k training steps, and for Places365 there were about 130k. For Circulant-STRING, block size 16 yielded the best results.

E.2. WebLI-3D Retrieval

All experiments were trained from scratch and used 256×256 image resolution. The ViT baseline used RGB data, but all RoPE and STRING variants used RGB-D, where we incorporated depth as a third coordinate for 3D position representations. We trained with batch size 8192 for 20 epochs, amounting to about 155k training steps using the SigLIP [52] pretraining setup. Training used the cosine decay learning rate schedule with 0.001 base learning rate and 5% warm-up steps (about 8k). For Circulant-STRING, block size 32 yielded the best results.

F. 3D Detection Details

F.1. Dataset

We train our synthetic datasets of indoor living room and cluttered tabletop scenes using a procedural generation recipe. We use open-sourced 3D assets, specifically a subset of assets from the Amazon Berkeley Objects [12] (ABO) dataset for background and tabletop objects and the YCB [4] and Google Scanned Objects [16] for tabletop clutter placement. The procedural generation recipe works by hierarchically generating sub-areas such as a lounge, dining, office and reading area within a randomly-sized rectangular room together with freestanding pieces of furniture, sampled from the different classes within the ABO assets. For scenes with tabletop clutter, we additionally randomly

sample assets on top of existing placement surfaces (e.g. tables) in the scene, and procedurally vary the packing fraction on top of the placement surface to achieve randomised clutter arrangements. We additionally vary lighting, background colors, camera extrinsics, intrinsics, aspect ratios etc to generate diverse datasets. Each example within a dataset has accompanied metadata such as RGB, Depth, Segmentation masks, Object poses, Camera extrinsics, intrinsics, and 3D & 2D bounding boxes for each object in view. We render images using Unity [47] achieving a high-degree of photo-realism.

We generated four different 1-million image datasets using the procedure above: a) living room scenes without tabletop clutter (Fig. 12a), b) tabletop scenes with procedurally varying clutter, placed within randomly created living room scenes (Fig. 12b), c) tabletop scenes with multiple cluttered table tops (Fig. 12c), and d) tabletop scenes with more complicated object arrangements, primarily objects placed on top of trays and other flat objects within the scene (Fig. 12d). We held out the first 20 images for each of these datasets for evaluation, and used the rest for training. Example images from our datasets showing the images and corresponding 3D bounding box labels can be seen in Fig. 13.

F.2. Implementation

We base our implementation for 3D bounding box detection described in Section 4.2.2 on OWL-ViT [32]. OWL-ViT predicts 2D, axis-aligned bounding boxes for the queried object names in the image. We modify this to predict full 3D bounding boxes. We utilized the vision and text towers from the SigLIP task in Section 4.1, and add the box and class prediction heads on top for the 3D bounding box task. The main differences with the original OWL-ViT are described in the sections below.

F.2.1. Box Format

The output of the box head in OWL-ViT is the relative offset from the corresponding image patch for the 4 edges (top, bottom, left, and right) of the axis-aligned, 2D bounding box. In 3D, image patches do not align with predictions quite as well as in 2D, so instead, the box head predicts the absolute pose of the 3D bounding box. The output format of the box head is

$$[< translation >, < rotation >, < size >]$$

where $< translation > \in \mathbb{R}^3$ is the SE(3) translation of the center of the box, $< rotation > \in \mathbb{R}^6$ is the first 2 columns of the SO(3) rotation matrix of the box, and $< size > \in \mathbb{R}^3$ is the length of each side of the box. Thus the output of the model for each predicted box is a 12D vector. All predictions are relative to the camera frame.

F.2.2. Loss Function

We modify the OWL-ViT loss terms in the following way. We keep the class loss as-is. We also keep the L1 loss directly on the 12D bounding box prediction vector. However, this loss is not necessarily optimal for SO(3) rotations and in future work we would like to look into better rotation losses such as [23]. Finally, we completely replace the 2D, axis-aligned intersection-over-union (IOU) loss. The algorithm for computing the full 3D, non-axis-aligned IOU is non-differentiable, so we instead compute a loss over the 8 corner vertices of the box. For both the predicted and target boxes, we compute the 3D coordinates of the 8 corners of the boxes, and then we sum the L1 distance between the predicted and target corners (we assume a fixed ordering of the corners). We found this loss significantly increases overall performance of the learned models.

F.2.3. 3D IOU

We use a Monte Carlo algorithm to compute the intersection-over-union (IOU) between 2 boxes in 3D space, which we report in our evaluation numbers in Section 4.2.2. First, we sample 100k 3D points uniformly at random inside the predicted box. Next, we transform those points to be in the coordinate space of the target box and we compute the ratio of points that are inside the target box. Finally, we compute IOU as the intersection of the volumes divided by the sum of the volumes minus the intersection:

$$IOU(box_p, box_t) = \frac{r * vol(box_p)}{vol(box_t) + (1 - r) * vol(box_p)}$$

where r is the fraction of sampled points inside the target box and vol is the volume of the given box.

F.2.4. Bipartite Matcher

We found empirically that the Hungarian algorithm [27] for bipartite matching between predictions and targets during training had the best performance. Specifically, the Hungarian algorithm was the only bipartite matching algorithm we tested that was able to correctly detect objects far from the center of the image. We suspect that, due to the absolute bounding box predictions described above in Appendix F.2.1, the box head was biased towards objects in the center of the camera frame, and the Hungarian algorithm encouraged it to predict boxes for further away objects. However, we also found the Hungarian algorithm to produce significant volatility in the performance of the model, with models sometimes getting stuck in local minima with poor performance during training (see Table 6). To counteract this, for every entry in Table 4 in Section 4.2.2, we trained 3 models with different random seeds. The values in the table are the max of the 3 models’ performance after 250k training iterations. Note that the parameters for the vision and text towers are loaded from the pre-trained checkpoints, so only the parameters for the prediction heads are randomly initialized.

F.3. Model and Training Configuration

The model is composed of 4 parts: the vision tower, the text tower, the class head, and the box head. The vision tower encodes the image as a set of tokens, the text tower encodes each input text sequence as a token, the class head predicts the class probabilities for the predictions given the query texts, and the box head outputs the bounding box parameters for each prediction. For the vision and text towers, we use the same model layout (e.g., number of layers) as the models trained on WebLI-3D in Section 4.1. For the class head we use the same layout as described in OWL-ViT [32]. We modify the box head to be a 3 layer MLP, with GELU [21] non-linearities after the first 2 layers. Unlike OWL-ViT, which outputs a relative offset for the bounding box, our box head directly outputs the absolute bounding box representation itself, as described above in Appendix F.2.1.

We use the same training method and optimizer as described in appendix A1.3 in [32], replacing the gIoU weight with a weight for our 8 corner vertex loss described above in Appendix F.2.2 but keeping the same values. To improve training speed, we randomly subsample 12 objects from each image for each iteration. However, during evaluation we include all objects in the scene when computing the 3D IOU. We train with a batch size of 1,024 for 250k iterations with an initial learning rate of $1e-4$.

F.4. Results From All Runs

For each configuration of RoPE/STRING and ViT/ViTD, we trained 3 models with different random seeds. In Table 4 in Section 4.2.2 we report the maximum IOU of the 3 for each configuration. Here, Table 6 shows the IOU for every run, with the maximum highlighted in bold. Note that for 3 runs (2

	Baseline	RoPE	RoPE-M	Circulant-S	Cayley-S
ViT	49.77	55.77	2.53	56.69	58.85
	49.10	52.11	57.17	58.95	58.43
	47.85	58.09	2.46	13.12	57.47
ViTD	65.88	71.21	70.78	72.36	70.36
	66.49	69.50	69.94	69.86	72.67
	67.60	70.31	70.90	68.51	71.91

Table 6 | Average 3D IOU % over all objects for the 3D bounding box prediction task. For each configuration, 3 models were trained with different random seeds. Baseline indicates no RoPE or STRING. The maximum for each configuration is highlighted in **bold**. Higher is better.

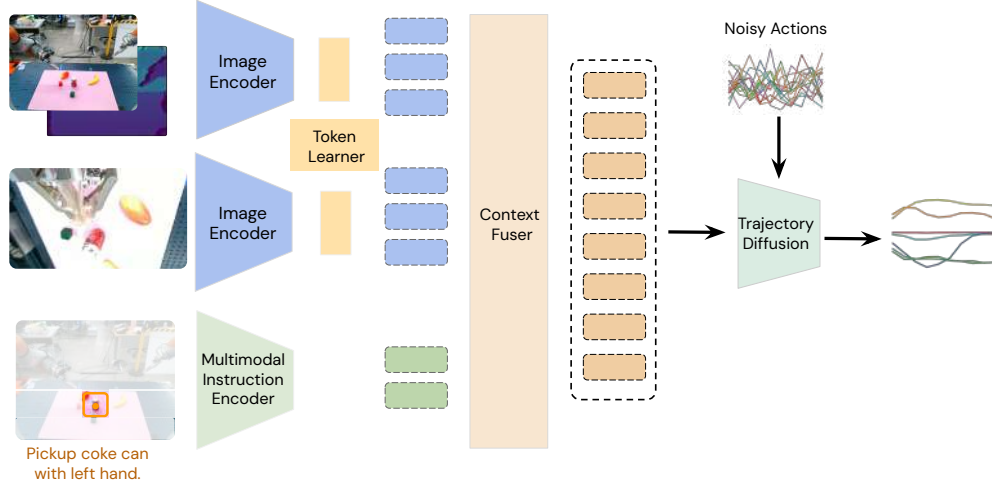


Figure 14 | Generative policy architecture for robotic manipulation using Kuka-IIWA arms.

for ViT+RoPE-M and 1 for ViT+Circulant-STRING), the models fell into a local minima which they never left and thus had inferior performance. See Appendix F.2.4 for details on possible causes.

G. Details of Generative Robotics Policies

Figure 14 shows the network architecture for the generative robotics policies for manipulation tasks. PaliGemma [2] VLM with embedding size 256 and patch size 16 is used for image encoding. The policy is trained with Adam optimizer with $1e-4$ learning rate and $1e-4$ weight decay. We use a linear learning rate warm-up for first 10000 steps of training. The policy is trained for a total of 500000 steps with batch size 256.